

Creative Ownership in the Age of AI

Annie Liang
(Northwestern)

Jay Lu
(UCLA)

ASU Theory Seminar

Introduction

Generative AI systems—trained on large corpora of human-authored works—are increasingly used to produce new creative content.

Introduction

Generative AI systems—trained on large corpora of human-authored works—are increasingly used to produce new creative content.

- Novels

The AI-generated Books Trend is Getting Worse

It has gotten so bad that Amazon's new policy limits self-published Kindle ebooks to three a day.



Sascha Brodsky, Contributor
January 24, 2024

🕒 4 Min Read

AI BUSINESS

Introduction

Generative AI systems—trained on large corpora of human-authored works—are increasingly used to produce new creative content.

- Novels
- Cartoons

Generating New Yorker-Style Cartoons with AI

Harnessing the power of OpenAI's ChatGPT to Transform Cartoon ideas into Reality



Georgiy Ermakov

Follow

5 min read · Oct 25, 2023

Introduction

Generative AI systems—trained on large corpora of human-authored works—are increasingly used to produce new creative content.

- Novels
- Cartoons
- Even AI “actors”

Tilly Norwood: how scared should we be of the viral AI ‘actor’?

A bunch of code is being pushed as the next Scarlett Johansson, a creation that is already causing pushback from real human actors

Tilly Norwood



A 2025 AI-generated image of Tilly Norwood, taken from the character's [Instagram](#) account

Introduction

Generative AI systems—trained on large corpora of human-authored works—are increasingly used to produce new creative content.

- Novels
- Cartoons
- Even AI “actors”

These creations can closely imitate style without copying content, a capability now central to ongoing litigation.

- Andersen v. Stability AI
 - Disney Enterprises v. Midjourney
 - Universal Music Group Recordings v. Suno
- + many others

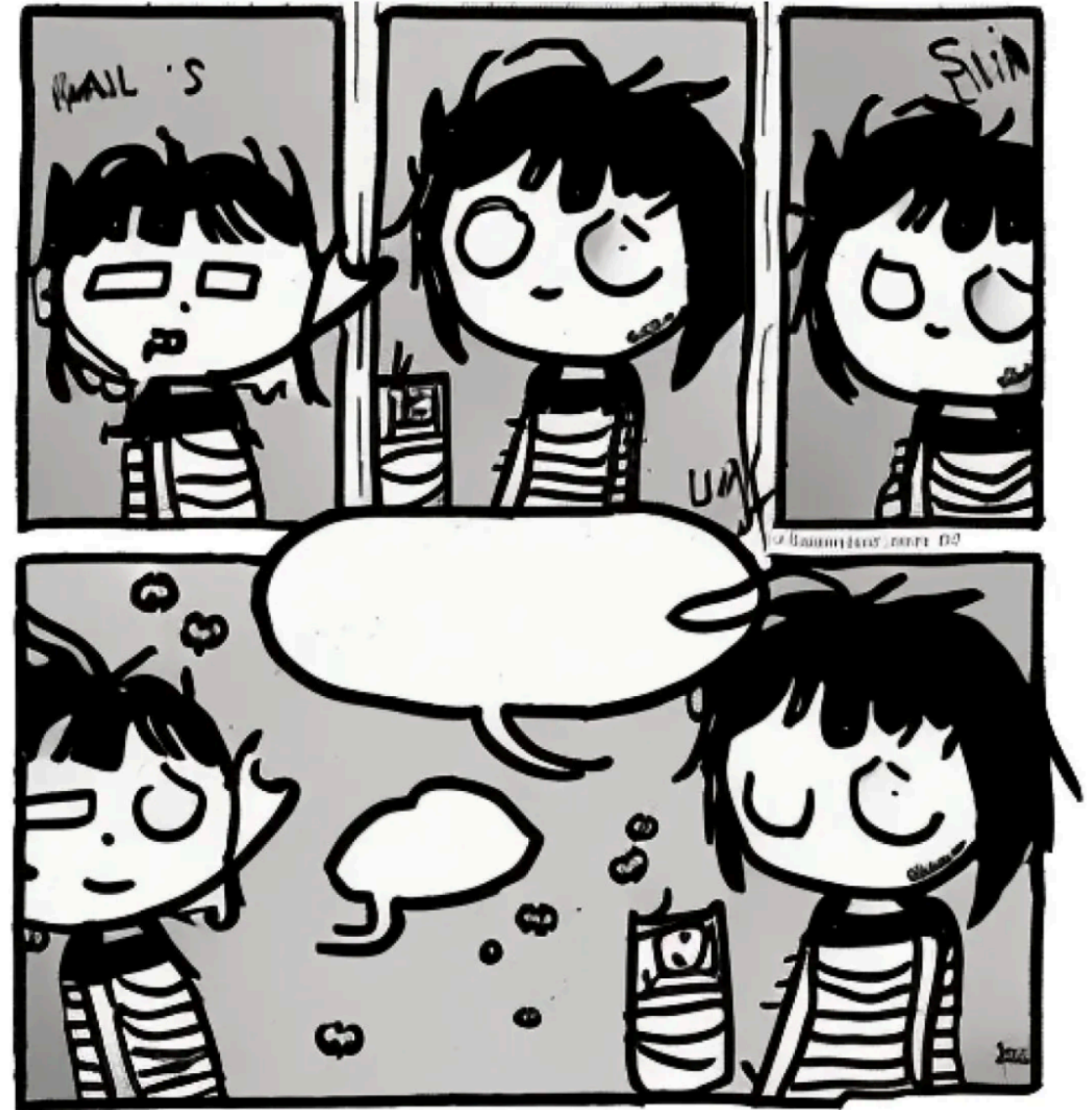
Introduction

A recent example of my work



Sarah Andersen, *NYTimes*, 2022

An A.I. generated image that I created when I used my name as a prompt



It's not perfect — but it has captured the signature elements of my drawing style. The black bangs, striped shirt and wide eyes are immediately recognizable. As A.I. gets increasingly sophisticated, I am growing more concerned about what's possible.

Legal Context

Existing copyright doctrine applies to GenAI at two stages:

1. **Training** on copyrighted works may violate copyright law.

- Possibly too strong: human authors learn by reading copyrighted books.
- May overly constrain AI capabilities to not allow it to learn in a similar way
- Moreover, fair use doctrine suggests that training on copyrighted works may be OK if the use is sufficiently “transformative”

2. **Generated outputs** may violate copyright law if they are “substantially similar” to an existing work.

- Copyright law has historically drawn a line between protectable **expression** and unprotectable **style** to preserve creative competition and follow-on innovation.
- But did not account for the emergence of AI systems that could easily, flexibly, quickly mimic style

This Paper

Existing notions of copyright and intellectual property **may not be appropriate for a post-generative AI world.**

This paper:

- We propose a new criterion for evaluating when an output is derivative of an existing work:

Would the output have been generable without that existing work in its training corpus?

- Study the implications of this criterion for permissible generation
- Whether the creative process is **light or heavy tailed** matters critically for the restrictiveness of this criterion

Model

Setup

- A *creation* is a vector $x \in \mathbb{R}^d$
- A *corpus* is any (Borel measurable) subset of creations $C \subset \mathbb{R}^d$
- Let $\mathcal{C}$ denote the set of all possible corpora
- A *generator* is a technology for producing new creations from an existing corpus.

Generators

Definition: The map $g : \mathcal{C} \rightarrow \mathcal{C}$ is a *generator* if for all $C, C' \in \mathcal{C}$ it satisfies:

1. Preservation: $C \subseteq g(C)$ — *reproduces all input creations*
2. Monotonicity: $C \subseteq C'$ implies $g(C) \subseteq g(C')$ — *weakly expands the set of generable creations when the original corpus expands*
3. Idempotence: $g(g(C)) = g(C)$ — *running it on its output produces no new creations*

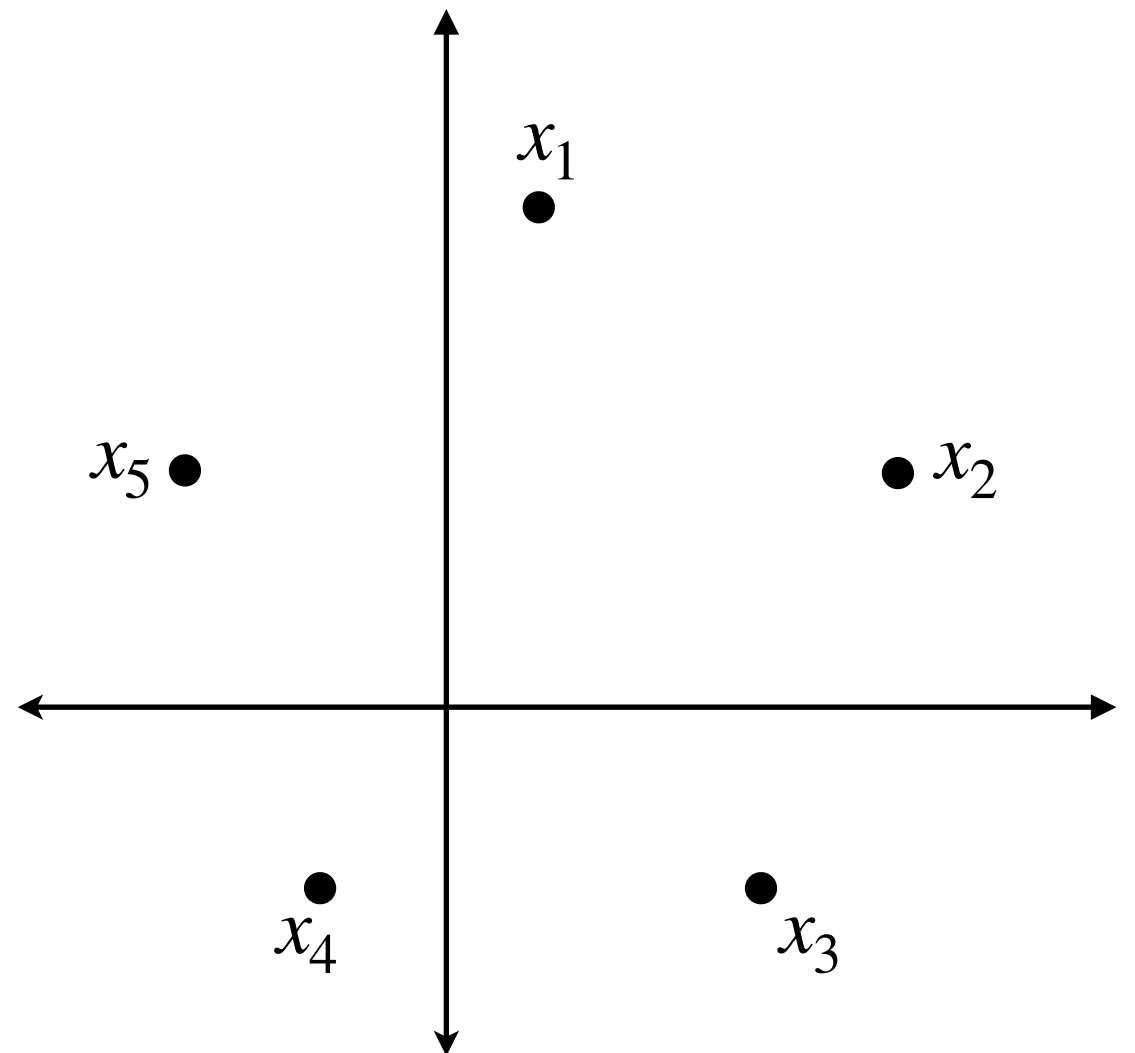
These are also known as closure operators.

We interpret $g(C)$ interpreted as the set of all possible creations given the initial training corpus C .

Example Generators

Convex Hull Generator: $g_{conv}(C) = \text{Conv}(C)$

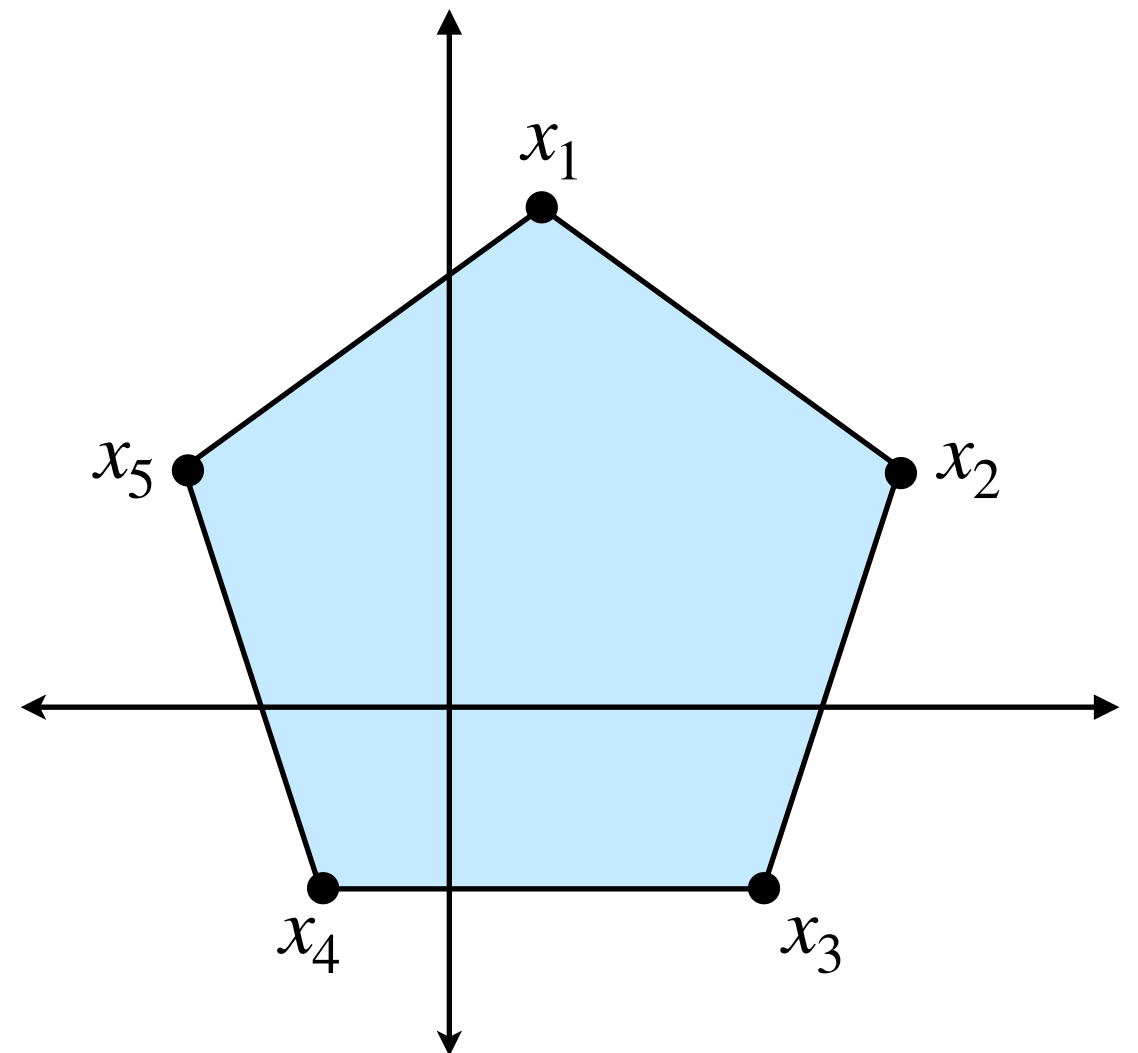
- Generate any convex combination of existing creations



Example Generators

Convex Hull Generator: $g_{conv}(C) = \text{Conv}(C)$

- Generate any convex combination of existing creations



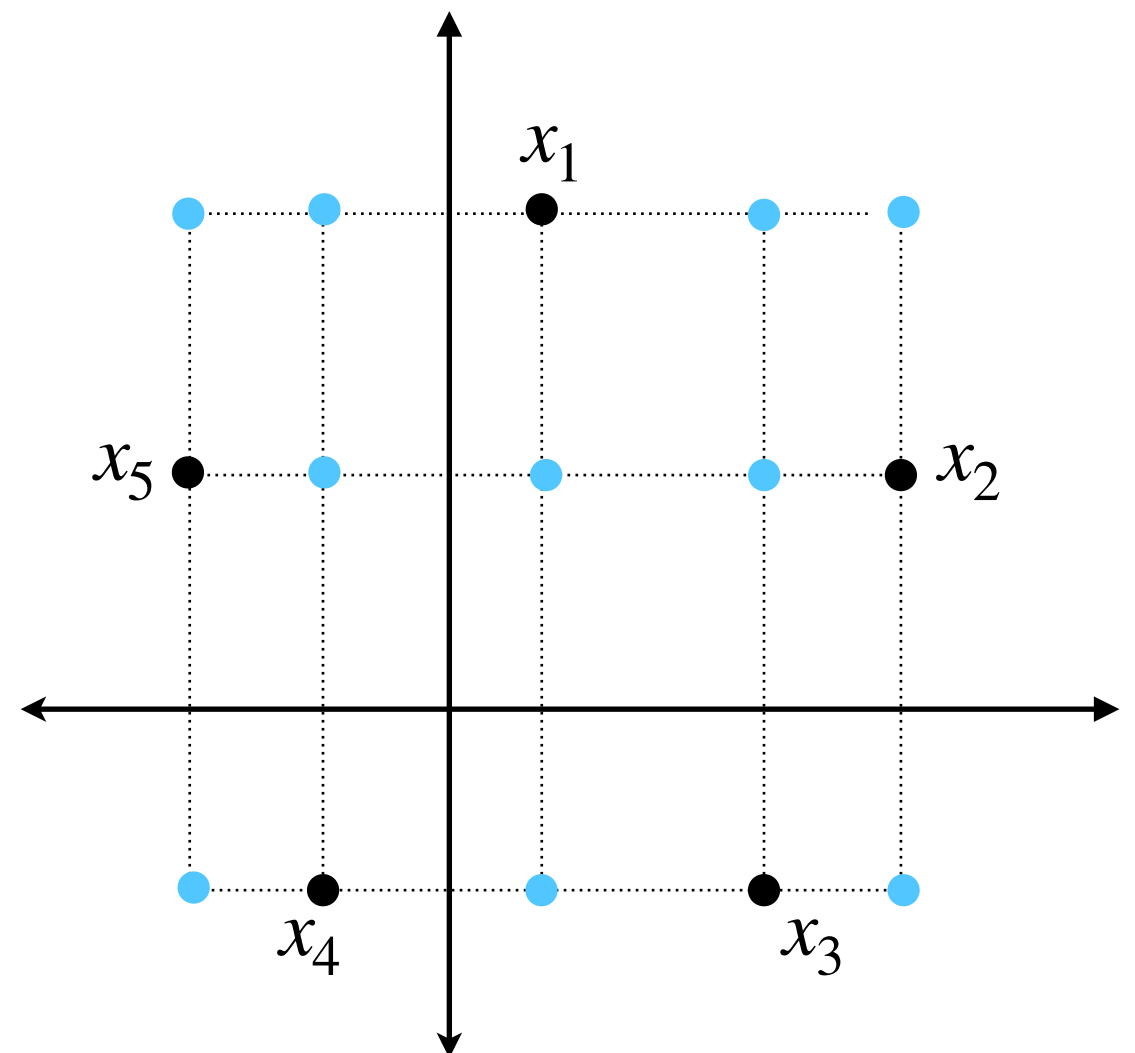
Example Generators

Convex Hull Generator: $g_{conv}(C) = \text{Conv}(C)$

- Generate any convex combination of existing creations

Splice Generator: $g_{splice}(C) := \{x \in \mathbb{R}^d : \forall k = 1, \dots, d, \text{ there is a } x' \in C \text{ s.t. } x_k = x'_k\}$.

- Recombines features of existing creations



Example Generators

Convex Hull Generator: $g_{conv}(C) = \text{Conv}(C)$

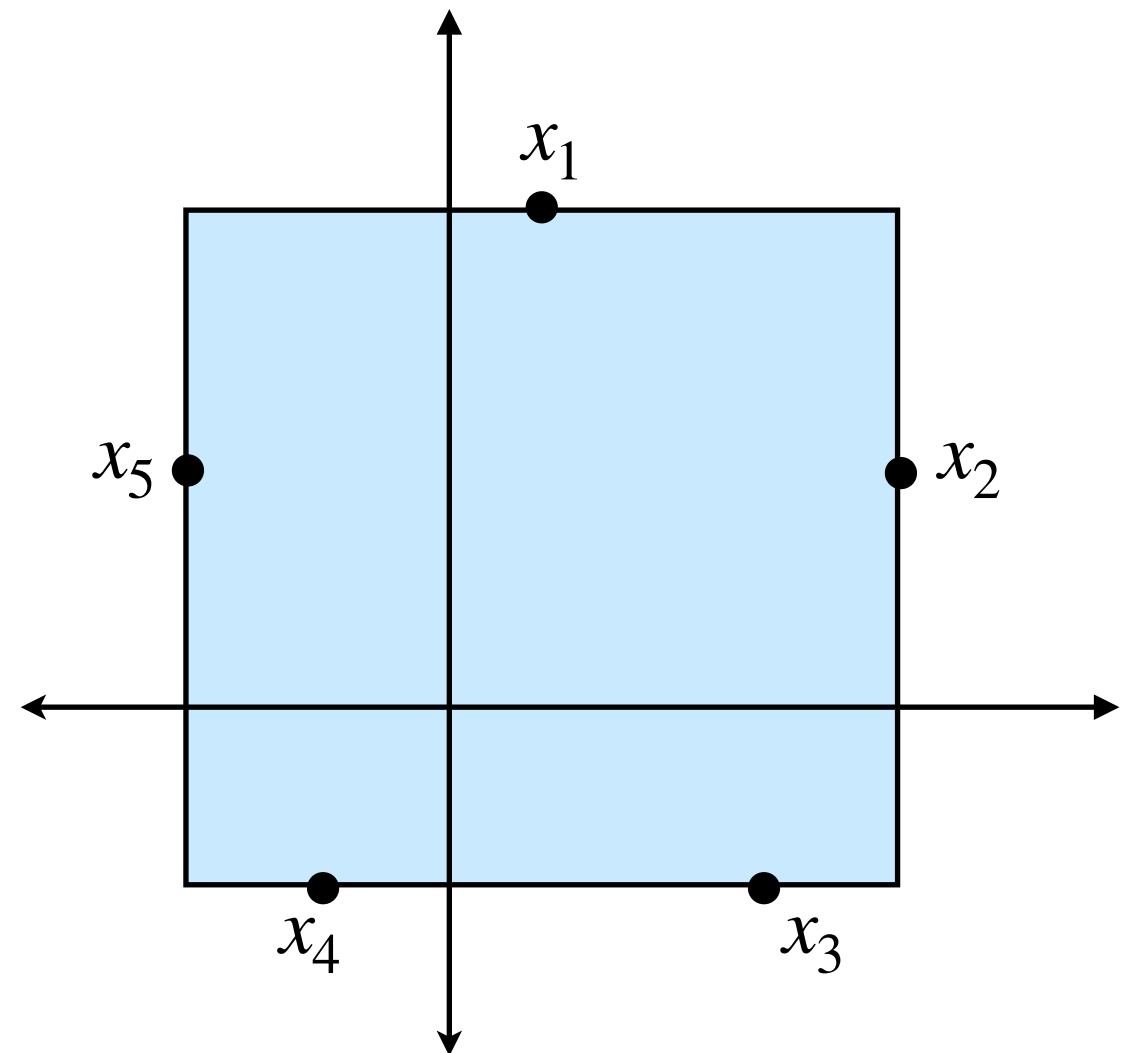
- Generate any convex combination of existing creations

Splice Generator: $g_{splice}(C) := \{x \in \mathbb{R}^d : \forall k = 1, \dots, d, \text{ there is a } x' \in C \text{ s.t. } x_k = x'_k\}$.

- Recombines features of existing creations

Box Generator: $g_{box}(C) = g_{conv}(g_{splice}(C))$

- Composes the convex hull and splice generator



Example Generators

Convex Hull Generator: $g_{conv}(C) = \text{Conv}(C)$

- Generate any convex combination of existing creations

Splice Generator: $g_{splice}(C) := \{x \in \mathbb{R}^d : \forall k = 1, \dots, d, \text{ there is a } x' \in C \text{ s.t. } x_k = x'_k\}$.

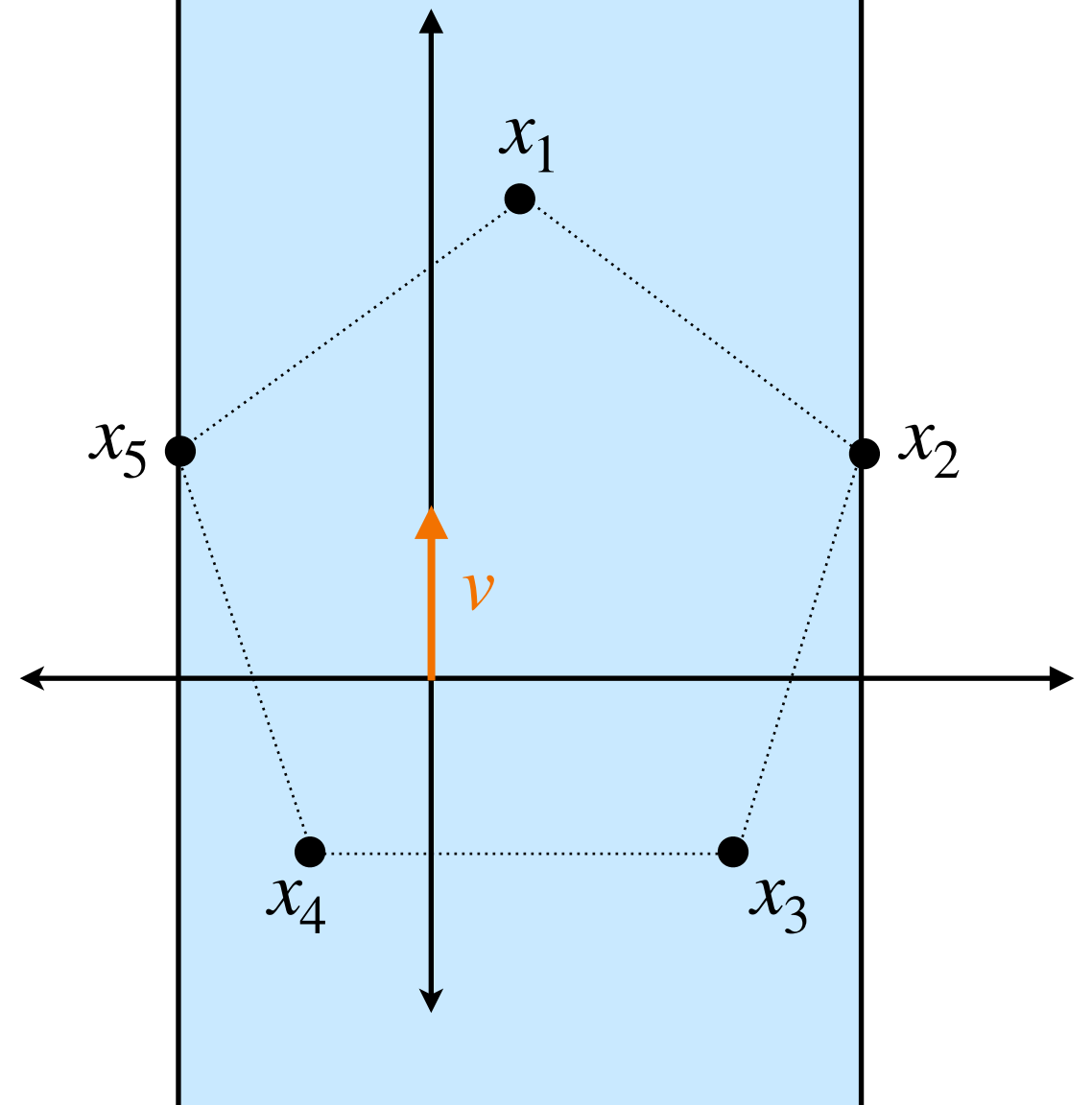
- Recombines features of existing creations

Box Generator: $g_{box}(C) = g_{conv}(g_{splice}(C))$

- Composes the convex hull and splice generator

v -Transform Generator:

- Shift the convex hull along a fixed direction v .



Convex-Valued Generators

Definition: A generator g is *convex-valued* if $g(C)$ is convex for every $C \in \mathcal{C}$.

- Includes the convex-hull generator, box generator, and the v -transform generator (but not the splice generator)

Result: The following are equivalent:

1. g is convex-valued
2. $g = g_{conv} \circ g$
3. $g = g \circ g_{conv}$

- Thus can interpret restriction to convex-valued generators as an assumption that the convex-hull generator is available
- Some of our results will make this restriction

Permissible Generation

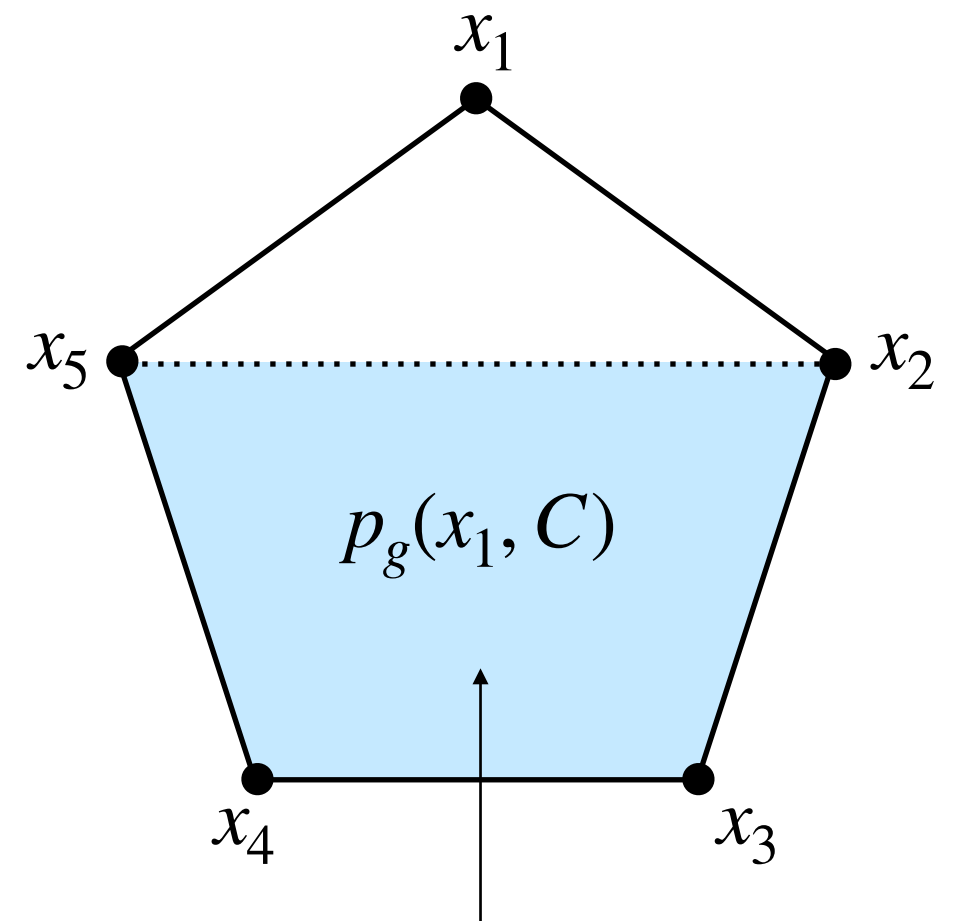
- Our proposed criterion specifies when a generated creation critically depends on the presence of some existing creation.
- Would the creation still be generable without that existing work in the corpus?

Permissible Generation

Definition: Fix a generator g and corpus C . For each existing creation $x \in C$:

1. The x -**permissible set** is $p_g(x, C) := g(C \setminus \{x\})$, i.e., all creations that are generable from $C \setminus \{x\}$
2. The x -**violation set** is $v_g(x, C) := g(C) \setminus p_g(x, C)$, i.e., all creations that are generable from C but not from $C \setminus \{x\}$

Convex hull generator:



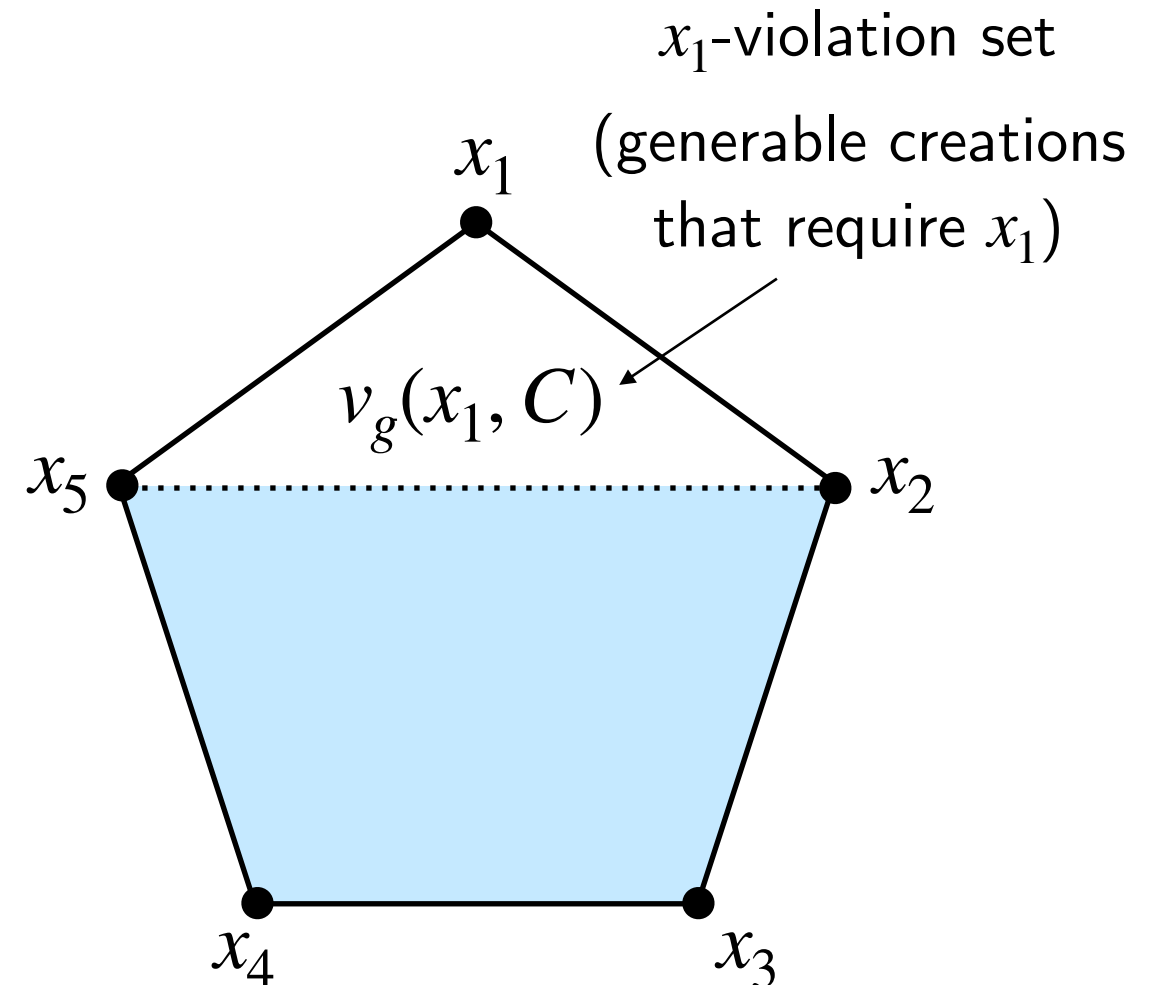
x_1 -permissible set
(generable creations that do not
require x_1)

Permissible Generation

Definition: Fix a generator g and corpus C . For each existing creation $x \in C$:

1. The x -**permissible set** is $p_g(x, C) := g(C \setminus \{x\})$, i.e., all creations that are generable from $C \setminus \{x\}$
2. The x -**violation set** is $v_g(x, C) := g(C) \setminus p_g(x, C)$, i.e., all creations that are generable from C but not from $C \setminus \{x\}$

Convex hull generator:



Permissible Generation

Definition:

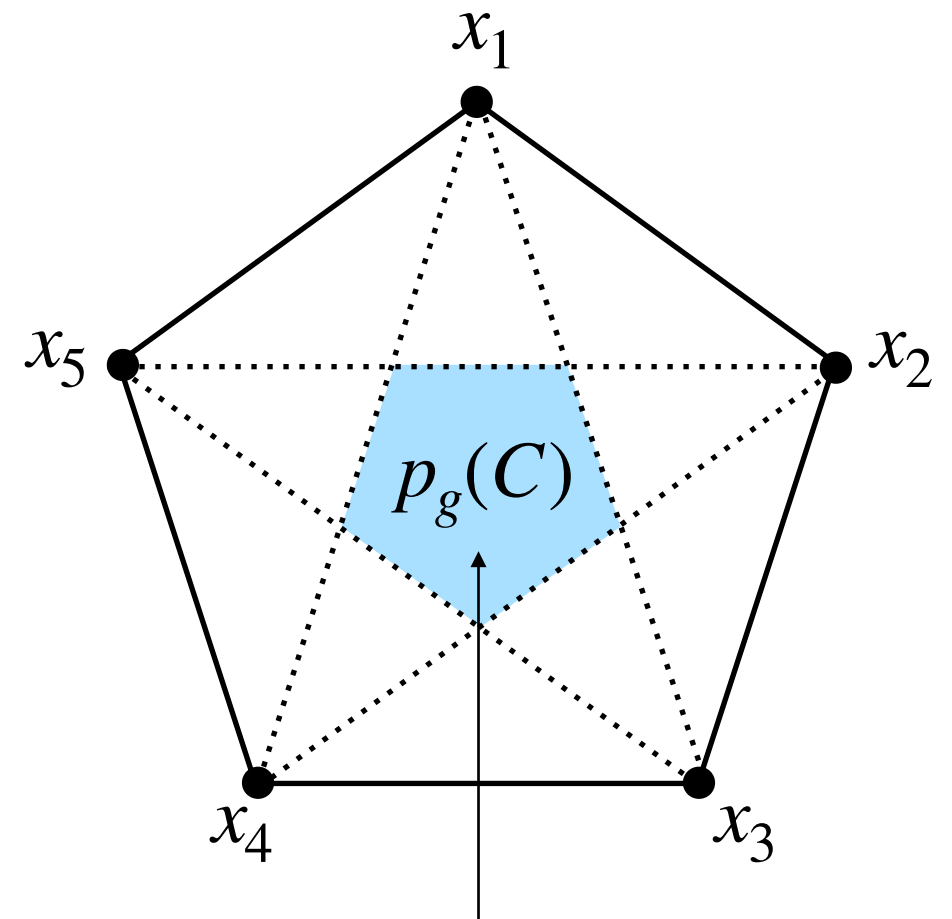
The **permissible set** is

$$p_g(C) := \bigcap_{x \in C} p_g(x, C)$$

The **violation set** is

$$v_g(C) := \bigcup_{x \in C} v_g(x, C)$$

Convex hull generator:



permissible set

(generable creations that do not critically rely on any $x \in C$)

Permissible Generation

Definition:

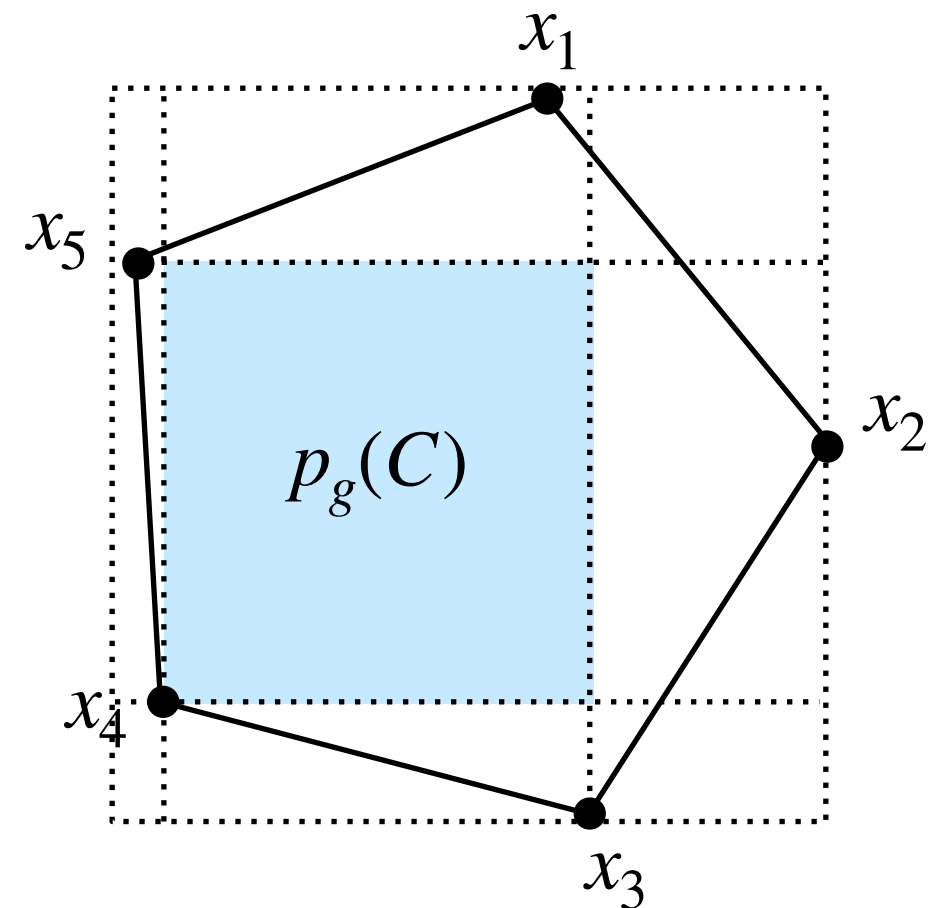
The **permissible set** is

$$p_g(C) := \bigcap_{x \in C} p_g(x, C)$$

The **violation set** is

$$v_g(C) := \bigcup_{x \in C} v_g(x, C)$$

Box generator:



Note: Boundary points **can** be permissible, depending on the generator and corpus

Interpretation

- If a generated creation lies in the **violation set**, then **at least one existing work** has a claim under our criterion.
- If a generated creation lies in the **permissible set**, then **no existing work** has such a claim.
- What does it mean for an existing work to “have a claim”?
 - The predecessor may be entitled to **attribution**.
 - The predecessor may be entitled to **financial compensation or rent**.
 - The downstream work may be **restricted or prohibited**.
- Our framework is agnostic about which, our goal is to understand which/how many generated outputs are permissible under the proposed criterion.

Restrictiveness

- Our criterion is not ranked in restrictiveness relative to the existing criteria (“no training on copyrighted works” + “substantial similarity” criterion)
- Sometimes more restrictive:
 - The violation set can generally be larger than the set of works that are “substantially similar” to some original
 - We restrict generation even if the AI system was not trained on any copyrighted works
- Sometimes less restrictive:
 - If your existing work was itself not original, we do not protect it
 - We do not restrict use of copyrighted works, as long as the generation is in the permissible set
- We view our criterion as a complement rather than a substitute for the existing criteria, could be that all should be simultaneously enforced

Implementation

- Brute-force retraining of an algorithm with the counterfactual training corpus is generally not feasible
- But the emerging literature of machine unlearning develops methods to approximate exactly this (Bourtoule et al., 2019; Xiong et al., 2024)
- Can also view the question of permissibility as an evidentiary question and compare outputs across independently trained algorithms

Properties of Permissible Creation

Results

1. The permissible set is monotone and self-stable.
2. Sufficient condition for non emptiness of the permissible set.
3. How the permissible set evolves with the addition of new works.

Results

1. The permissible set is monotone and self-stable.
2. Sufficient condition for non emptiness of the permissible set.
3. How the permissible set evolves with the addition of new works.

Monotonicity and self-stability

Result: The permissible set is

(a) **monotone:** $C \subseteq D \implies p_g(C) \subseteq p_g(D)$

i.e., adding new works to the corpus weakly expands the set of permissible generations.

(b) **self-stable:** $g(p_g(C)) = p_g(C)$

i.e., the permissible set is closed under the generator, so that generating from permissible works yields only permissible works

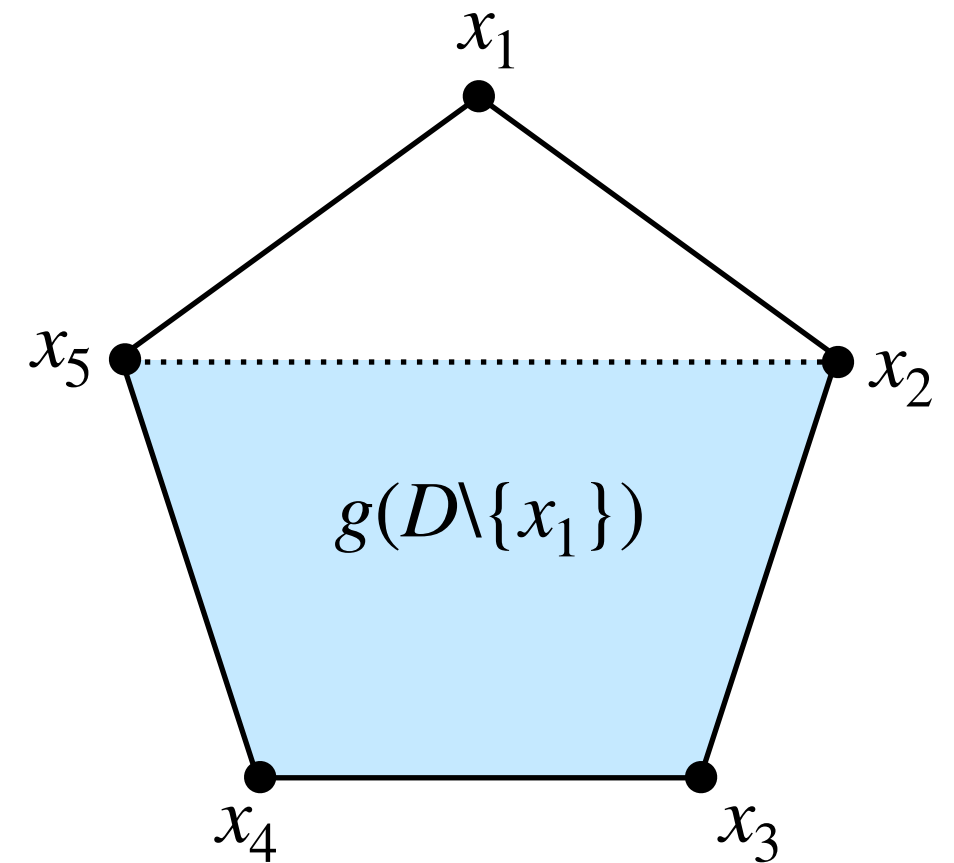
Proof of monotonicity

Result: $C \subseteq D \implies p_g(C) \subseteq p_g(D)$

Fix any $D \supseteq C$. Then

$$p_g(D) = \bigcap_{x \in D} g(D \setminus \{x\}) \quad (\text{by definition})$$

$$D = \{x_1, x_2, \dots, x_5\}$$



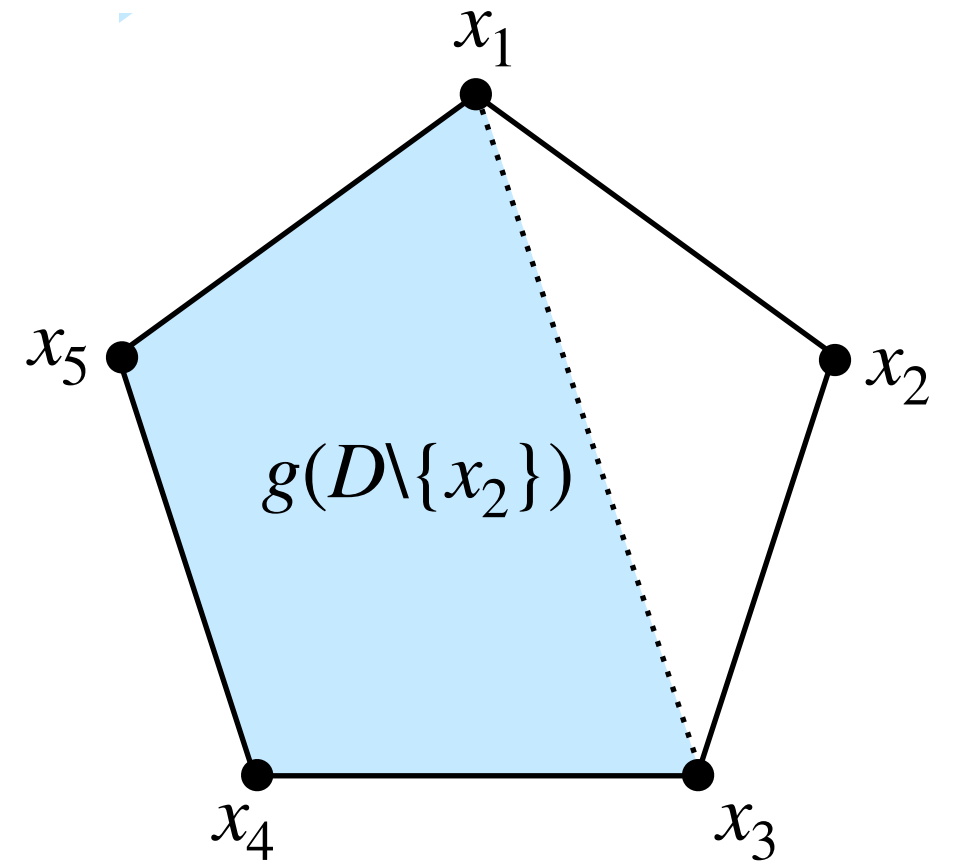
Proof of monotonicity

Result: $C \subseteq D \implies p_g(C) \subseteq p_g(D)$

Fix any $D \supseteq C$. Then

$$p_g(D) = \bigcap_{x \in D} g(D \setminus \{x\}) \quad (\text{by definition})$$

$$D = \{x_1, x_2, \dots, x_5\}$$



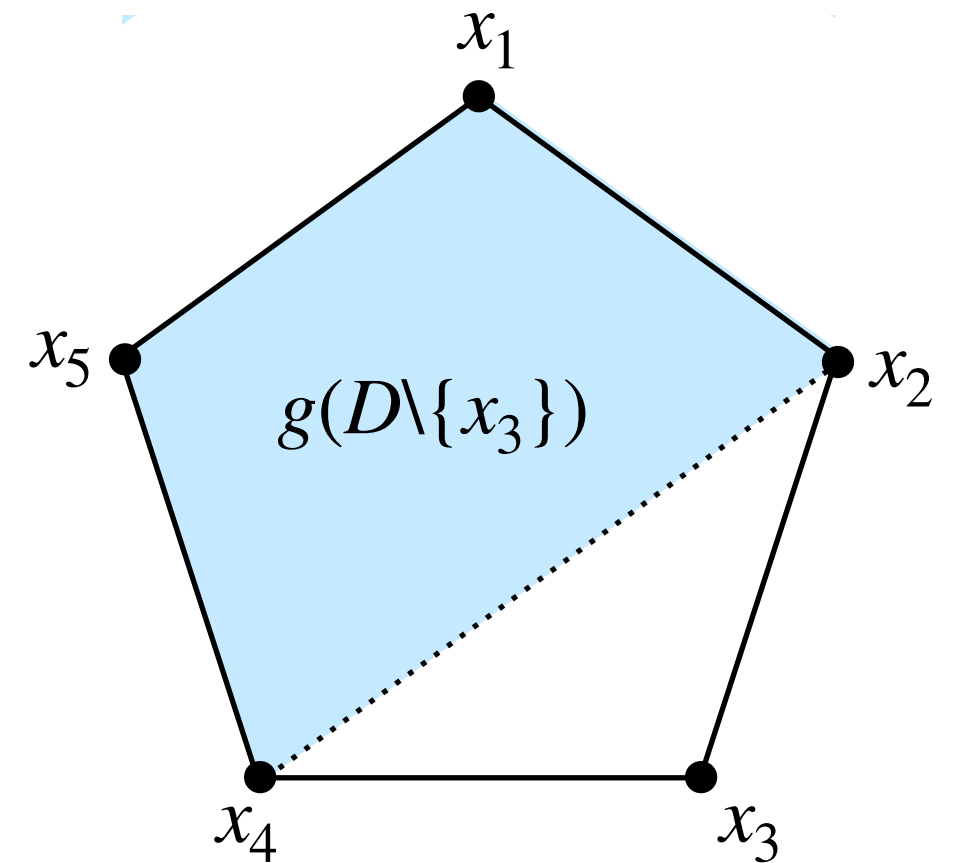
Proof of monotonicity

Result: $C \subseteq D \implies p_g(C) \subseteq p_g(D)$

Fix any $D \supseteq C$. Then

$$p_g(D) = \bigcap_{x \in D} g(D \setminus \{x\}) \quad (\text{by definition})$$

$$D = \{x_1, x_2, \dots, x_5\}$$



Proof of monotonicity

Result: $C \subseteq D \implies p_g(C) \subseteq p_g(D)$

Fix any $D \supseteq C$. Then

$$\begin{aligned} p_g(D) &= \bigcap_{x \in D} g(D \setminus \{x\}) \quad (\text{by definition}) \\ &= \bigcap_{x \in C} g(D \setminus \{x\}) \cap \bigcap_{x' \in D \setminus C} g(D \setminus \{x'\}) \end{aligned}$$

Proof of monotonicity

Result: $C \subseteq D \implies p_g(C) \subseteq p_g(D)$

Fix any $D \supseteq C$. Then

$$\begin{aligned} p_g(D) &= \bigcap_{x \in D} g(D \setminus \{x\}) \quad (\text{by definition}) \\ &= \bigcap_{x \in C} g(D \setminus \{x\}) \cap \bigcap_{x' \in D \setminus C} g(D \setminus \{x'\}) \end{aligned}$$

Recall that

$$p_g(C) = \bigcap_{x \in C} g(C \setminus \{x\})$$

Proof of monotonicity

Result: $C \subseteq D \implies p_g(C) \subseteq p_g(D)$

Fix any $D \supseteq C$. Then

$$\begin{aligned} p_g(D) &= \bigcap_{x \in D} g(D \setminus \{x\}) \quad (\text{by definition}) \\ &= \bigcap_{x \in C} g(D \setminus \{x\}) \cap \bigcap_{x' \in D \setminus C} g(D \setminus \{x'\}) \end{aligned}$$

Recall that

$$p_g(C) = \bigcap_{x \in C} g(C \setminus \{x\})$$

Then the inclusion $p_g(C) \subseteq p_g(D)$ follows from:

- ① $g(C \setminus \{x\}) \subseteq g(D \setminus \{x\})$
 - ② $g(C \setminus \{x\}) \subseteq g(D \setminus \{x'\})$
- $\forall x \in C, x' \in D \setminus C$

Proof of monotonicity

$$\text{Result: } C \subseteq D \implies p_g(C) \subseteq p_g(D)$$

Fix any $D \supseteq C$. Then

$$\begin{aligned} p_g(D) &= \bigcap_{x \in D} g(D \setminus \{x\}) \quad (\text{by definition}) \\ &= \bigcap_{x \in C} g(D \setminus \{x\}) \cap \bigcap_{x' \in D \setminus C} g(D \setminus \{x'\}) \end{aligned}$$

Recall that

$$p_g(C) = \bigcap_{x \in C} g(C \setminus \{x\})$$

Then the inclusion $p_g(C) \subseteq p_g(D)$ follows from:

$$\begin{aligned} \textcircled{1} \quad & g(C \setminus \{x\}) \subseteq g(D \setminus \{x\}) \\ \textcircled{2} \quad & g(C \setminus \{x\}) \subseteq g(D \setminus \{x'\}) \end{aligned} \quad \forall x \in C, x' \in D \setminus C$$

$$\textcircled{1} \quad g(C \setminus \{x\}) \subseteq g(D \setminus \{x\})$$

Pf. Immediate from monotonicity of g

Proof of monotonicity

Result: $C \subseteq D \implies p_g(C) \subseteq p_g(D)$

Fix any $D \supseteq C$. Then

$$\begin{aligned} p_g(D) &= \bigcap_{x \in D} g(D \setminus \{x\}) \quad (\text{by definition}) \\ &= \bigcap_{x \in C} g(D \setminus \{x\}) \cap \bigcap_{x' \in D \setminus C} g(D \setminus \{x'\}) \end{aligned}$$

Recall that

$$p_g(C) = \bigcap_{x \in C} g(C \setminus \{x\})$$

Then the inclusion $p_g(C) \subseteq p_g(D)$ follows from:

$$\textcircled{1} \quad g(C \setminus \{x\}) \subseteq g(D \setminus \{x\})$$

$$\textcircled{2} \quad g(C \setminus \{x\}) \subseteq g(D \setminus \{x'\})$$

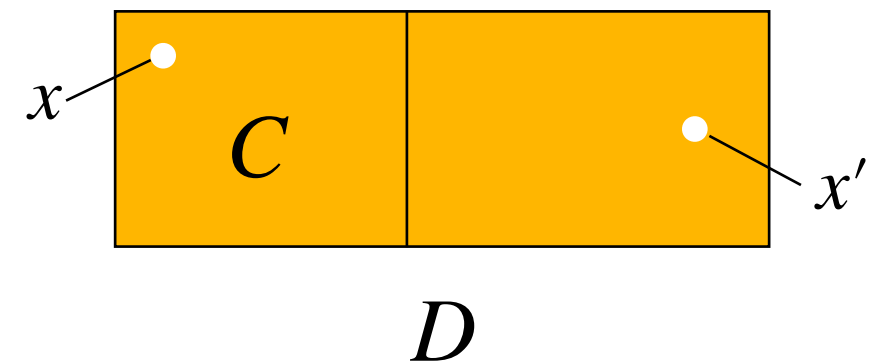
$$\forall x \in C, x' \in D \setminus C$$

$$\textcircled{1} \quad g(C \setminus \{x\}) \subseteq g(D \setminus \{x\})$$

Pf. Immediate from monotonicity of g

$$\textcircled{2} \quad g(C \setminus \{x\}) \subseteq g(D \setminus \{x'\})$$

Pf. Again by monotonicity of g , since $C \setminus \{x\} \subseteq D \setminus \{x'\}$



Proof of self-stability

Result: $p_g(C) = g(p_g(C))$

$p_g(C) \subseteq g(p_g(C))$ by **preservation**

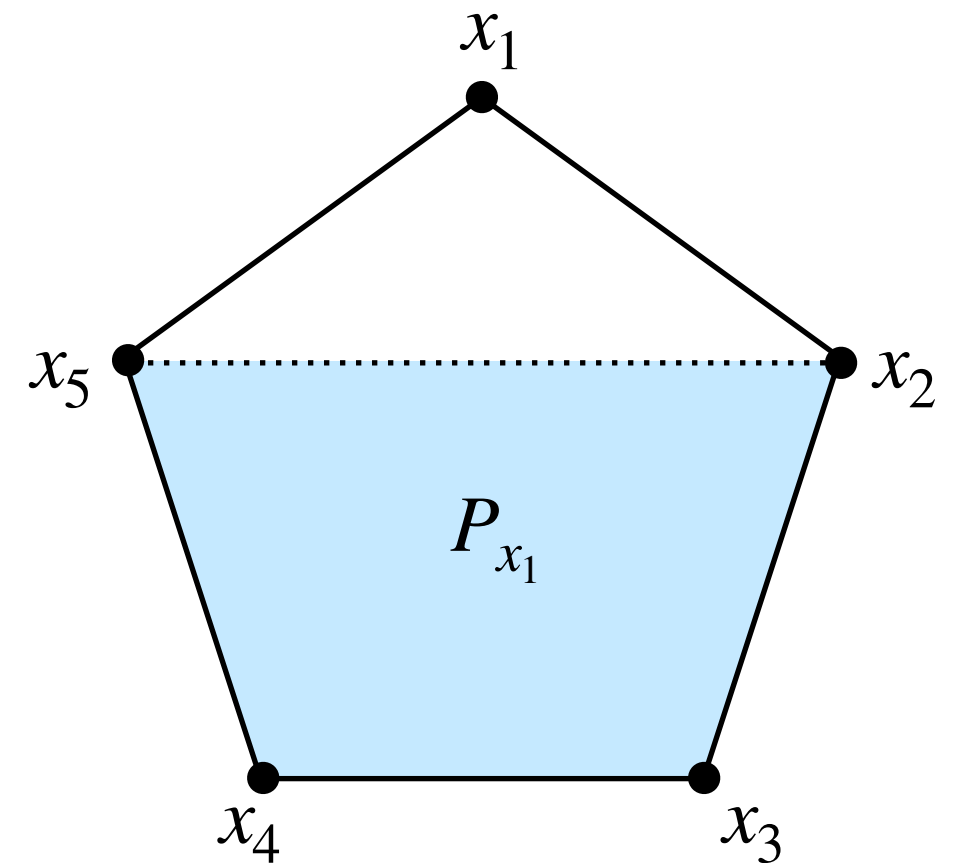
Proof of self-stability

Result: $p_g(C) = g(p_g(C))$

$p_g(C) \subseteq g(p_g(C))$ by **preservation**

To show: $g(p_g(C)) \subseteq p_g(C)$

Define $P_x = g(C \setminus \{x\})$ for the generations that can be created without x



Proof of self-stability

Result: $p_g(C) = g(p_g(C))$

$p_g(C) \subseteq g(p_g(C))$ by **preservation**

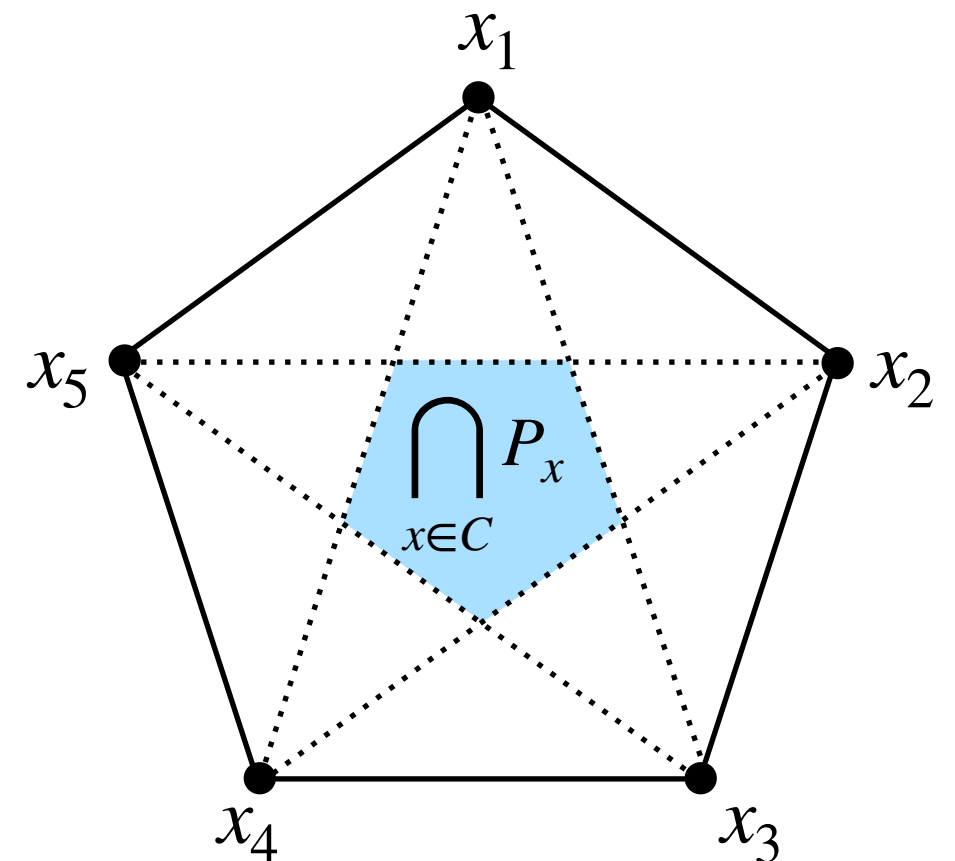
To show: $g(p_g(C)) \subseteq p_g(C)$

Define $P_x = g(C \setminus \{x\})$ for the generations that can be created without x

Since $\bigcap_{x \in C} P_x \subseteq P_x$ for every $x \in C$, also

$$g\left(\bigcap_{x \in C} P_x\right) \subseteq g(P_x) \quad \forall x \in C$$

by **monotonicity** of g



Proof of self-stability

Result: $p_g(C) = g(p_g(C))$

$p_g(C) \subseteq g(p_g(C))$ by **preservation**

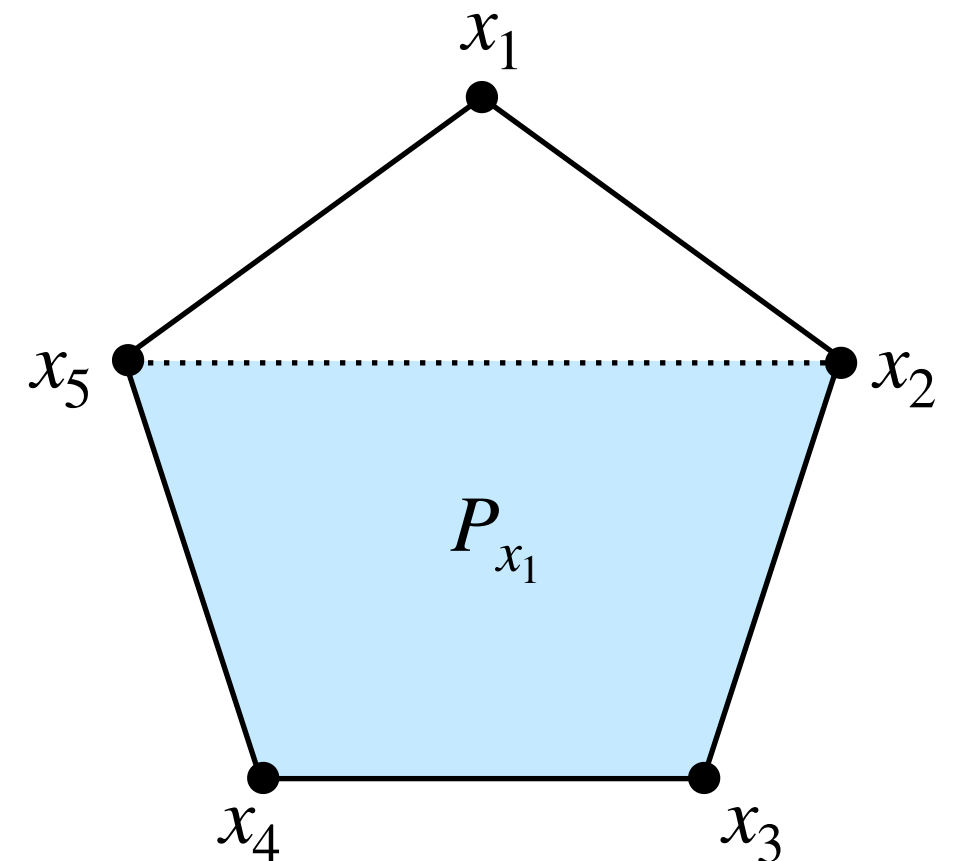
To show: $g(p_g(C)) \subseteq p_g(C)$

Define $P_x = g(C \setminus \{x\})$ for the generations that can be created without x

Since $\bigcap_{x \in C} P_x \subseteq P_x$ for every $x \in C$, also

$$g\left(\bigcap_{x \in C} P_x\right) \subseteq g(P_x) \quad \forall x \in C$$

by **monotonicity** of g



Proof of self-stability

Result: $p_g(C) = g(p_g(C))$

$p_g(C) \subseteq g(p_g(C))$ by **preservation**

To show: $g(p_g(C)) \subseteq p_g(C)$

Define $P_x = g(C \setminus \{x\})$ for the generations that can be created without x

Since $\bigcap_{x \in C} P_x \subseteq P_x$ for every $x \in C$, also

$$g\left(\bigcap_{x \in C} P_x\right) \subseteq g(P_x) \quad \forall x \in C \quad \Rightarrow \quad g\left(\bigcap_{x \in C} P_x\right) \subseteq \bigcap_{x \in C} g(P_x)$$

Proof of self-stability

Result: $p_g(C) = g(p_g(C))$

$p_g(C) \subseteq g(p_g(C))$ by **preservation**

To show: $g(p_g(C)) \subseteq p_g(C)$

Define $P_x = g(C \setminus \{x\})$ for the generations that can be created without x

Since $\bigcap_{x \in C} P_x \subseteq P_x$ for every $x \in C$, also

$$g\left(\bigcap_{x \in C} P_x\right) \subseteq g(P_x) \quad \forall x \in C \quad \implies \quad g\left(\bigcap_{x \in C} P_x\right) \subseteq \bigcap_{x \in C} g(P_x)$$

But for each $x \in C$,

$$\begin{aligned} g(P_x) &= g(g(C \setminus \{x\})) && \text{by definition of } P_x \\ &= g(C \setminus \{x\}) && \text{by idempotence} \end{aligned}$$

Proof of self-stability

Result: $p_g(C) = g(p_g(C))$

$p_g(C) \subseteq g(p_g(C))$ by **preservation**

To show: $g(p_g(C)) \subseteq p_g(C)$

Define $P_x = g(C \setminus \{x\})$ for the generations that can be created without x

Since $\bigcap_{x \in C} P_x \subseteq P_x$ for every $x \in C$, also

$$g\left(\bigcap_{x \in C} P_x\right) \subseteq g(P_x) \quad \forall x \in C \quad \implies \quad g\left(\bigcap_{x \in C} P_x\right) \subseteq \bigcap_{x \in C} g(P_x)$$

But for each $x \in C$,

$$\begin{aligned} g(P_x) &= g(g(C \setminus \{x\})) && \text{by definition of } P_x \\ &= g(C \setminus \{x\}) && \text{by idempotence} \end{aligned}$$

$$\begin{aligned} &= \bigcap_{x \in C} g(C \setminus \{x\}) \\ &= p_g(C) \end{aligned}$$

Proof of self-stability

Result: $p_g(C) = g(p_g(C))$

$p_g(C) \subseteq g(p_g(C))$ by **preservation**

To show: $g(p_g(C)) \subseteq p_g(C)$

Define $P_x = g(C \setminus \{x\})$ for the generations that can be created without x

$$\begin{aligned} g(p_g(C)) &= g\left(\bigcap_{x \in C} P_x\right) \subseteq \bigcap_{x \in C} g(P_x) \\ &= \bigcap_{x \in C} g(C \setminus \{x\}) \\ &= p_g(C) \end{aligned}$$

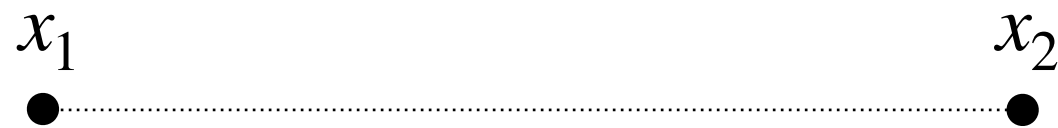
Results

1. The permissible set is monotone and self-stable.
2. Sufficient condition for non emptiness of the permissible set.
3. How the permissible set evolves with the addition of new works.

Nonemptiness of the permissible set

The permissible set can be empty.

Example: The generator is the convex hull generator, and the corpus consists of two works:



The x_1 -permissible set is $\{x_2\}$, and the x_2 -permissible set is $\{x_1\}$, so their intersection is empty.

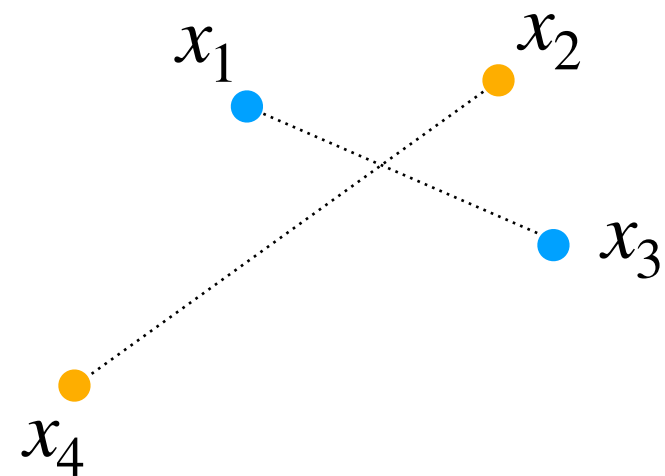
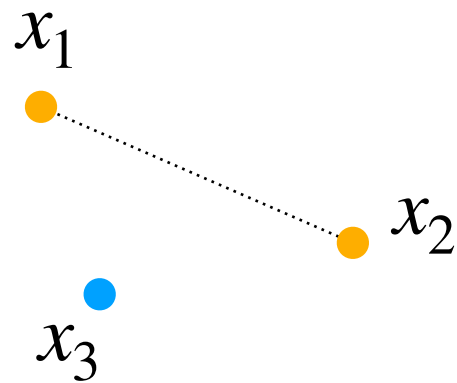
No generations are permissible.

When is the permissible set non-empty?

Definition: The *Radon number* of a generator g , denoted $R(g)$, is the smallest integer k such that any corpus C of size at least k contains two disjoint and nonempty subsets $A, B \subseteq C$ satisfying:

$$g(A) \cap g(B) \neq \emptyset$$

If no k exists, set $R(g) = \infty$.



Example: for the convex hull generator g_{conv} in $\mathbb{R}^d$, Radon's theorem implies $R(g_{conv}) = d + 2$.

When is the permissible set non-empty?

Result: Fix any generator g . If $|C| \geq R(g)$ then the permissible set is nonempty: $p_g(C) \neq \emptyset$.

Corollary: Fix any convex-valued generator g in $\mathbb{R}^d$. If $|C| \geq d + 2$ then the permissible set is nonempty: $p_g(C) \neq \emptyset$.

Results

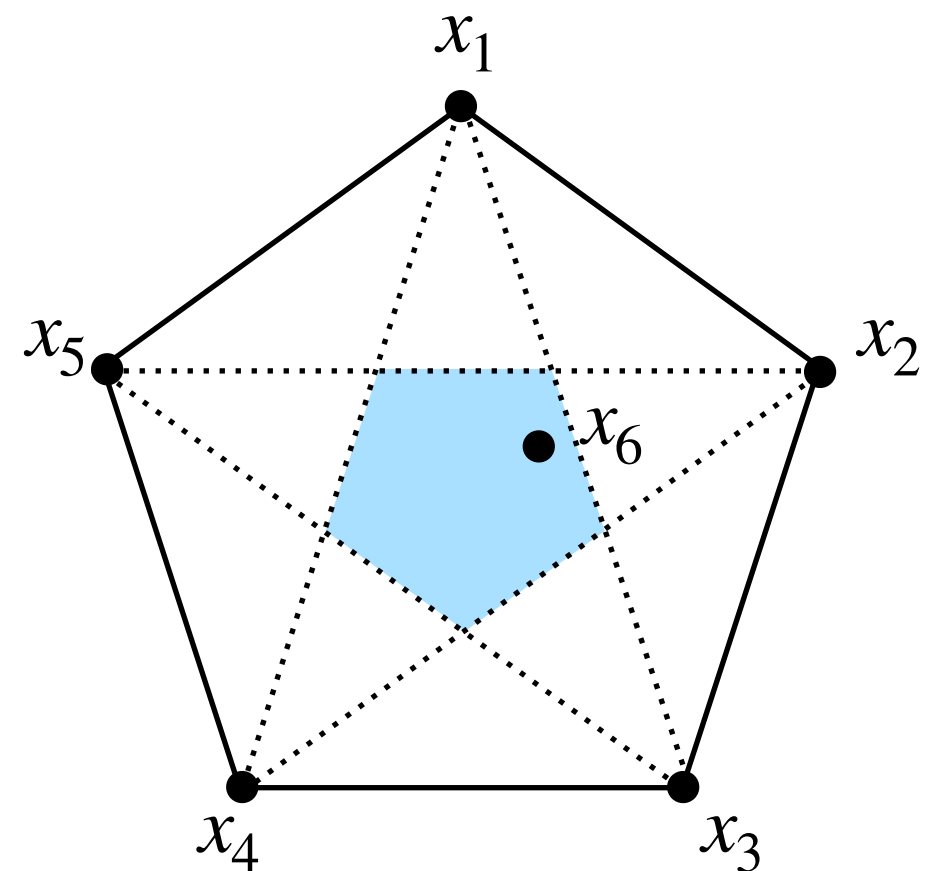
1. The permissible set is monotone and self-stable.
2. Sufficient condition for non emptiness of the permissible set.
3. How the permissible set evolves with the addition of new works.

What is the effect of expanding the corpus?

Result: Suppose $D = C \cup \{x\}$. Then:

1. If x is permissible, the permissible set is unchanged:

$$x \in p_g(C) \implies p_g(C) = p_g(D)$$



What is the effect of expanding the corpus?

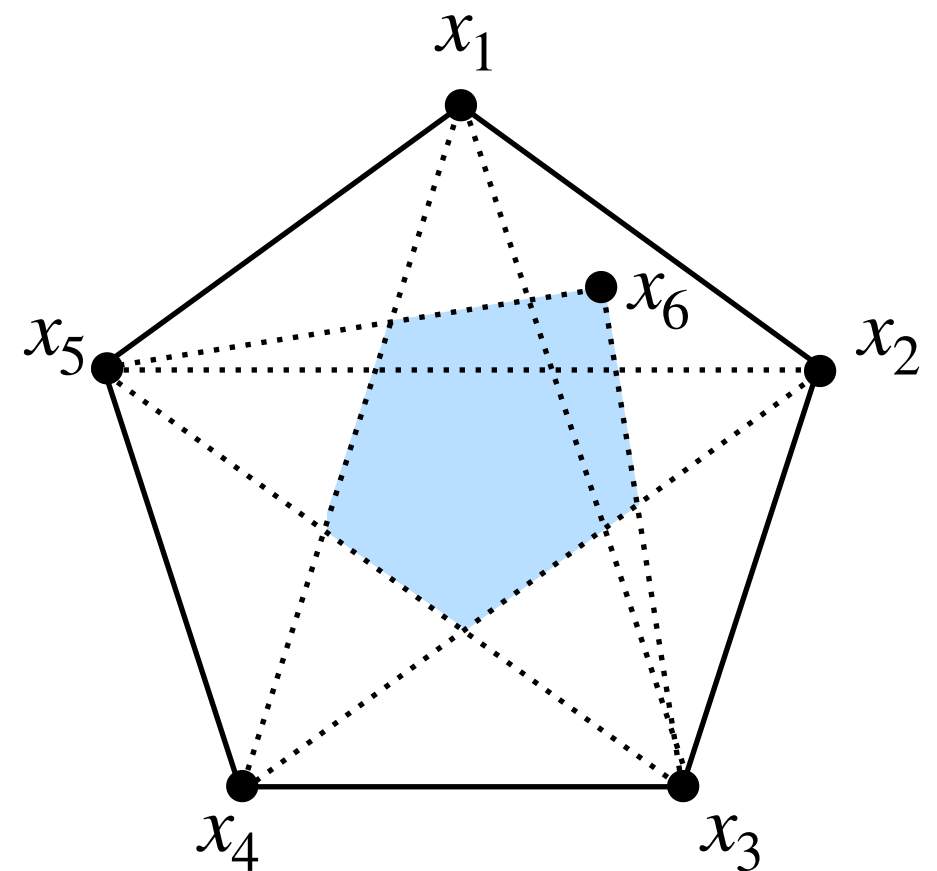
Result: Suppose $D = C \cup \{x\}$. Then:

1. If x is permissible, the permissible set is unchanged:

$$x \in p_g(C) \implies p_g(C) = p_g(D)$$

2. If x is a violation, adding it strictly expands the permissible set:

$$x \in v_g(C) \implies p_g(C) \subsetneq p_g(D)$$



What is the effect of expanding the corpus?

Result: Suppose $D = C \cup \{x\}$. Then:

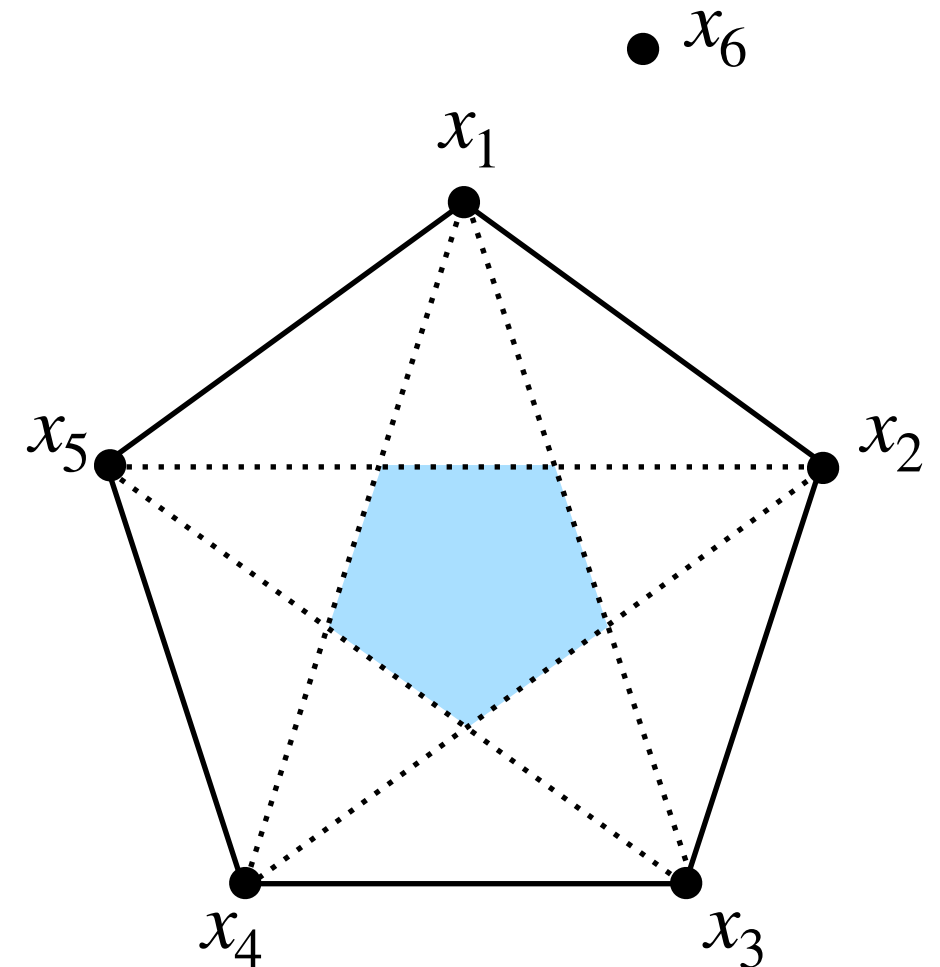
1. If x is permissible, the permissible set is unchanged:

$$x \in p_g(C) \implies p_g(C) = p_g(D)$$

2. If x is a violation, adding it strictly expands the permissible set:

$$x \in v_g(C) \implies p_g(C) \subsetneq p_g(D)$$

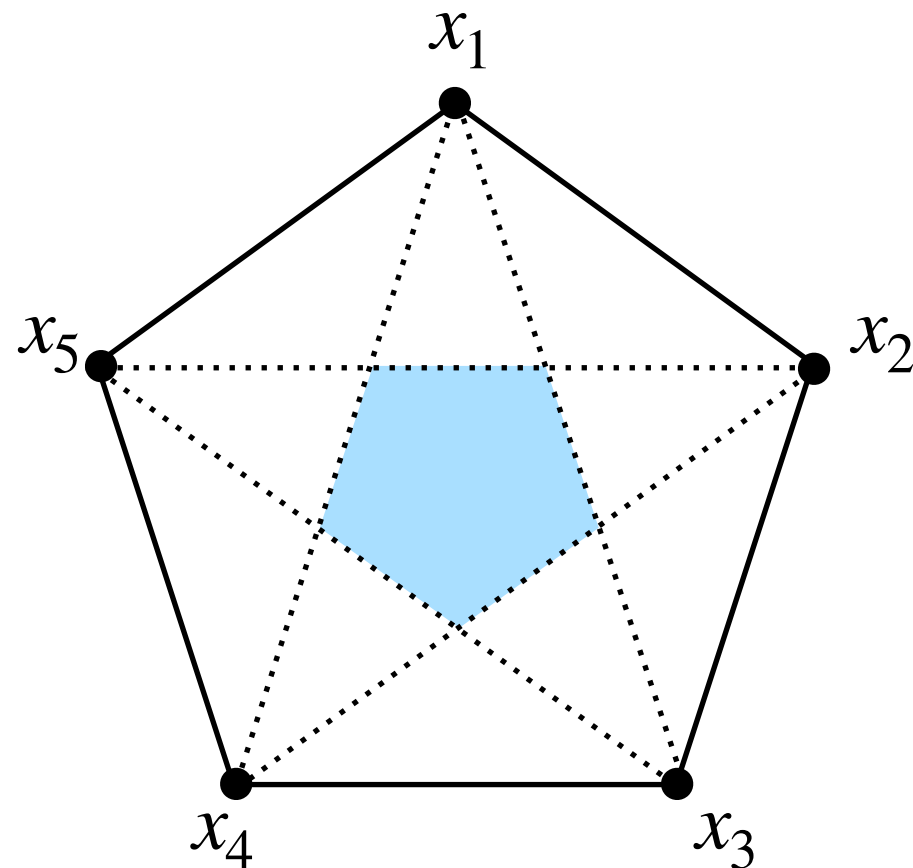
3. If x is not generable (i.e., $x \notin g(C)$), then both equality and strict expansion are possible.



Permissible Generation with Large Corpora

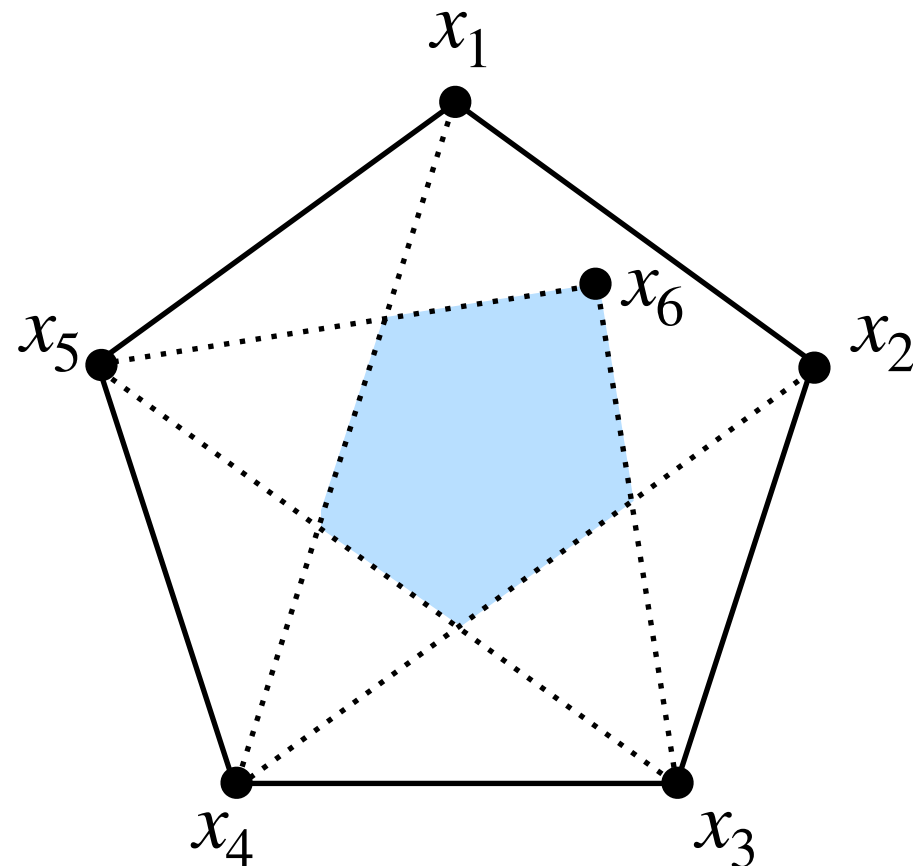
Long-Run Permissibility

Implication: Given a sufficiently large corpus of works from $g(C)$, eventually all generation is permissible.



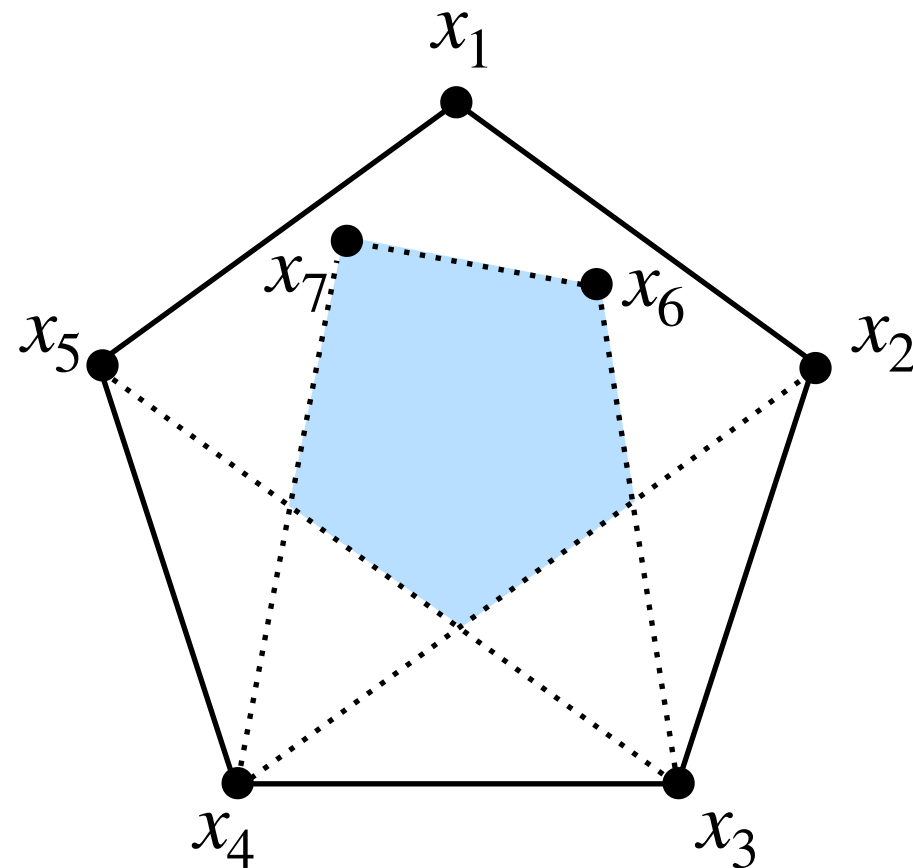
Long-Run Permissibility

Implication: Given a sufficiently large corpus of works from $g(C)$, eventually all generation is permissible.



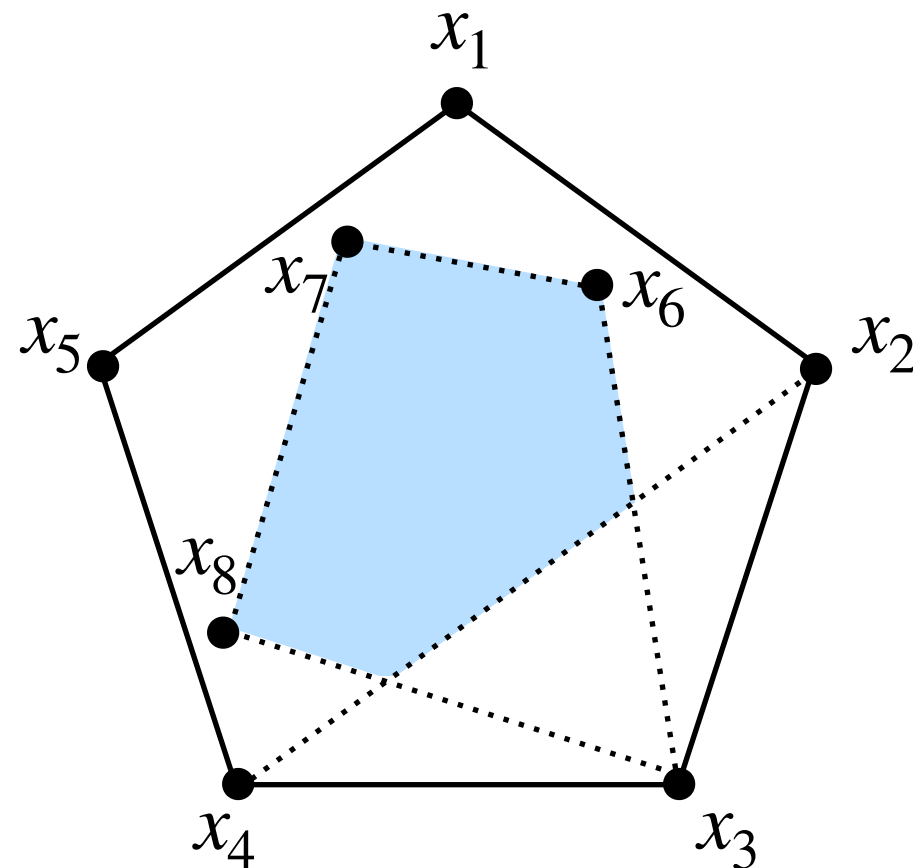
Long-Run Permissibility

Implication: Given a sufficiently large corpus of works from $g(C)$, eventually all generation is permissible.



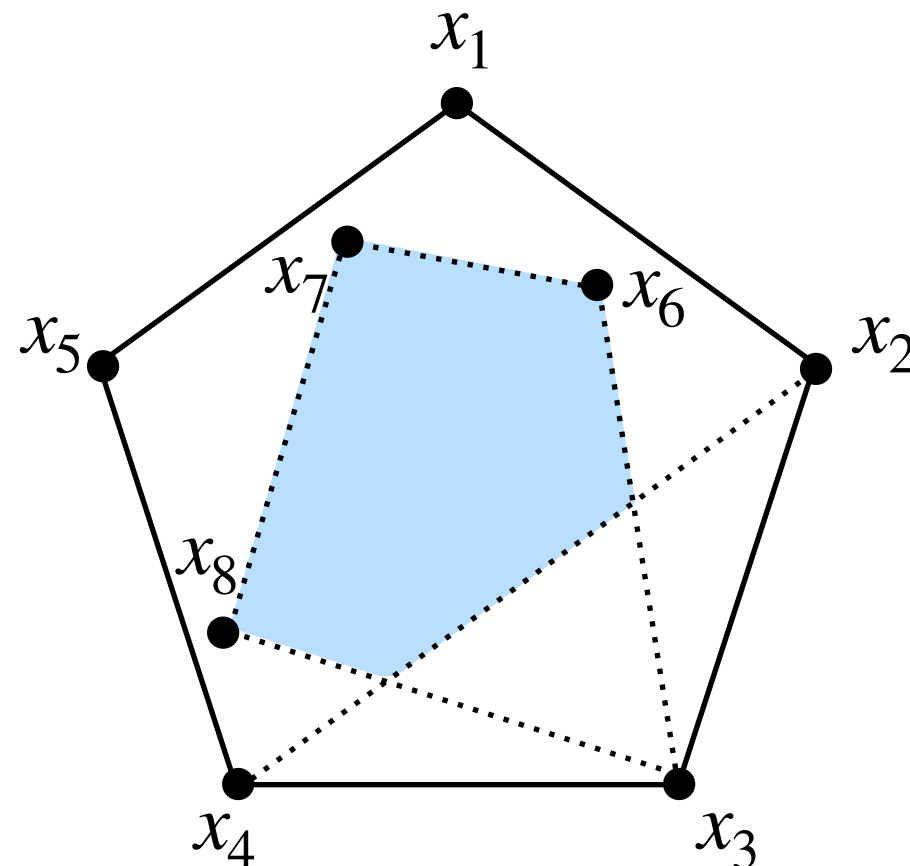
Long-Run Permissibility

Implication: Given a sufficiently large corpus of works from $g(C)$, eventually all generation is permissible.



Long-Run Permissibility

Implication: Given a sufficiently large corpus of works from $g(C)$, eventually all generation is permissible.



But **human** (or human-AI) generation may extend beyond $g(C)$.

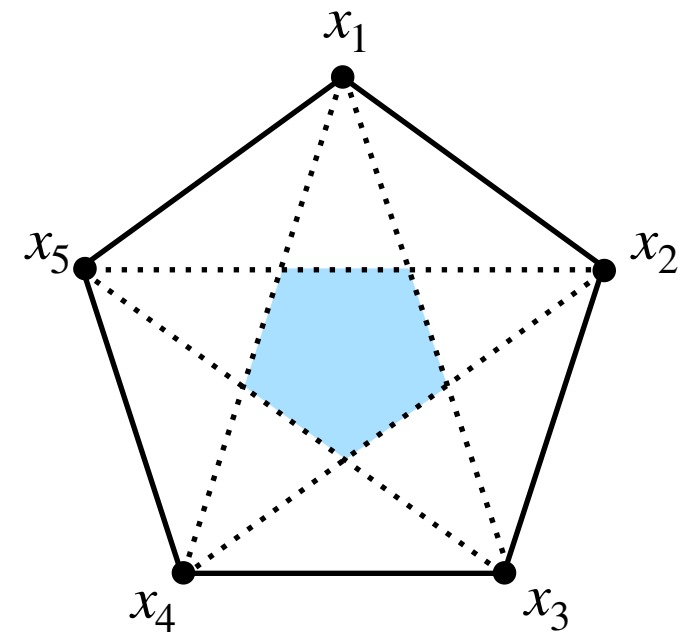
Next ask how typical permissible generation is given a large corpus of creative works.

Notion of Typicality

Let $\text{Vol}(A)$ denote the Lebesgue measure (or *volume*) of the set $A \in \mathcal{C}$.

Definition: The **permissible ratio** is:

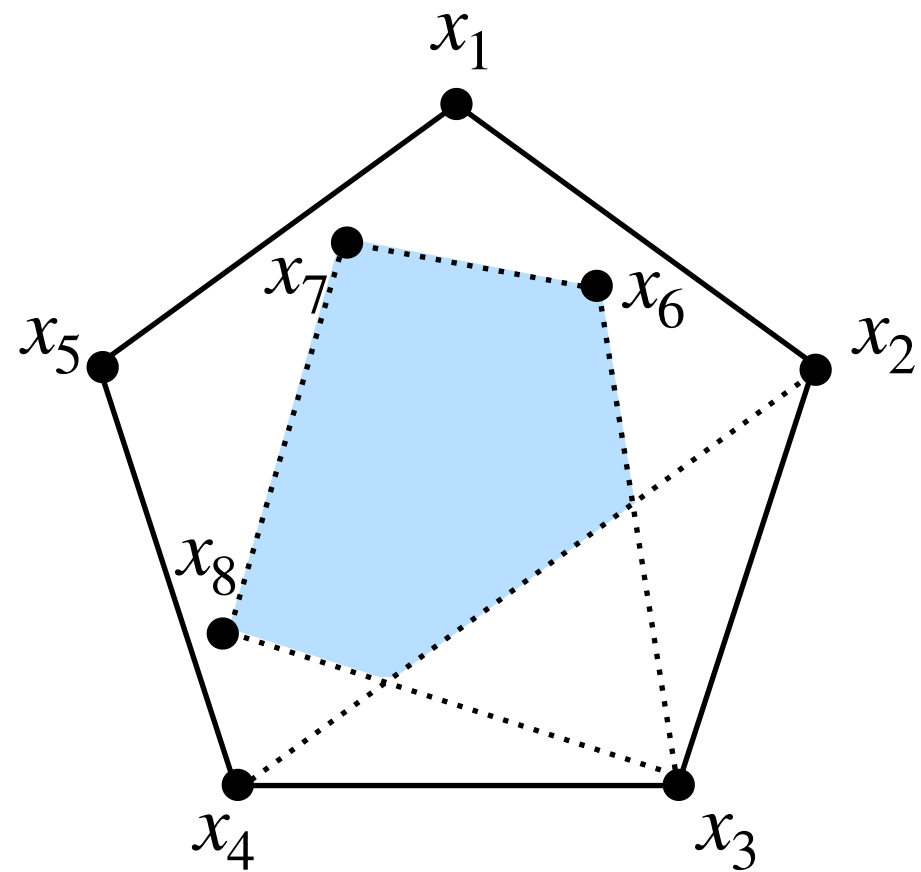
$$r_g(C) := \frac{\text{Vol}(p_g(C))}{\text{Vol}(g(C))}$$



i.e., the proportion (by volume) of generable creations that are permissible.

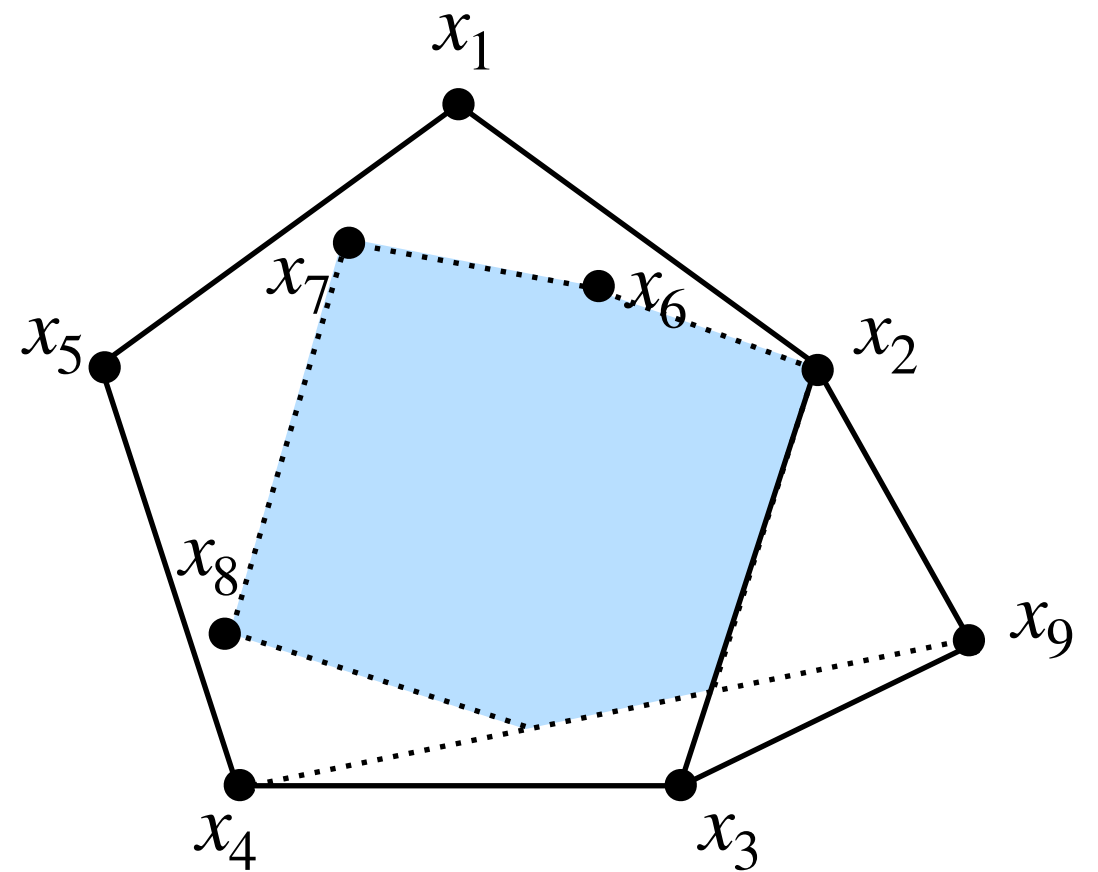
The Creative Process

- Let X_i be an i.i.d. random variable on $\mathbb{R}^d$
- For each $n \in \mathbb{N}$, define the random corpus $C_n = \{X_1, \dots, X_n\}$
- We want to understand $\lim_{n \rightarrow \infty} r_g(C_n)$



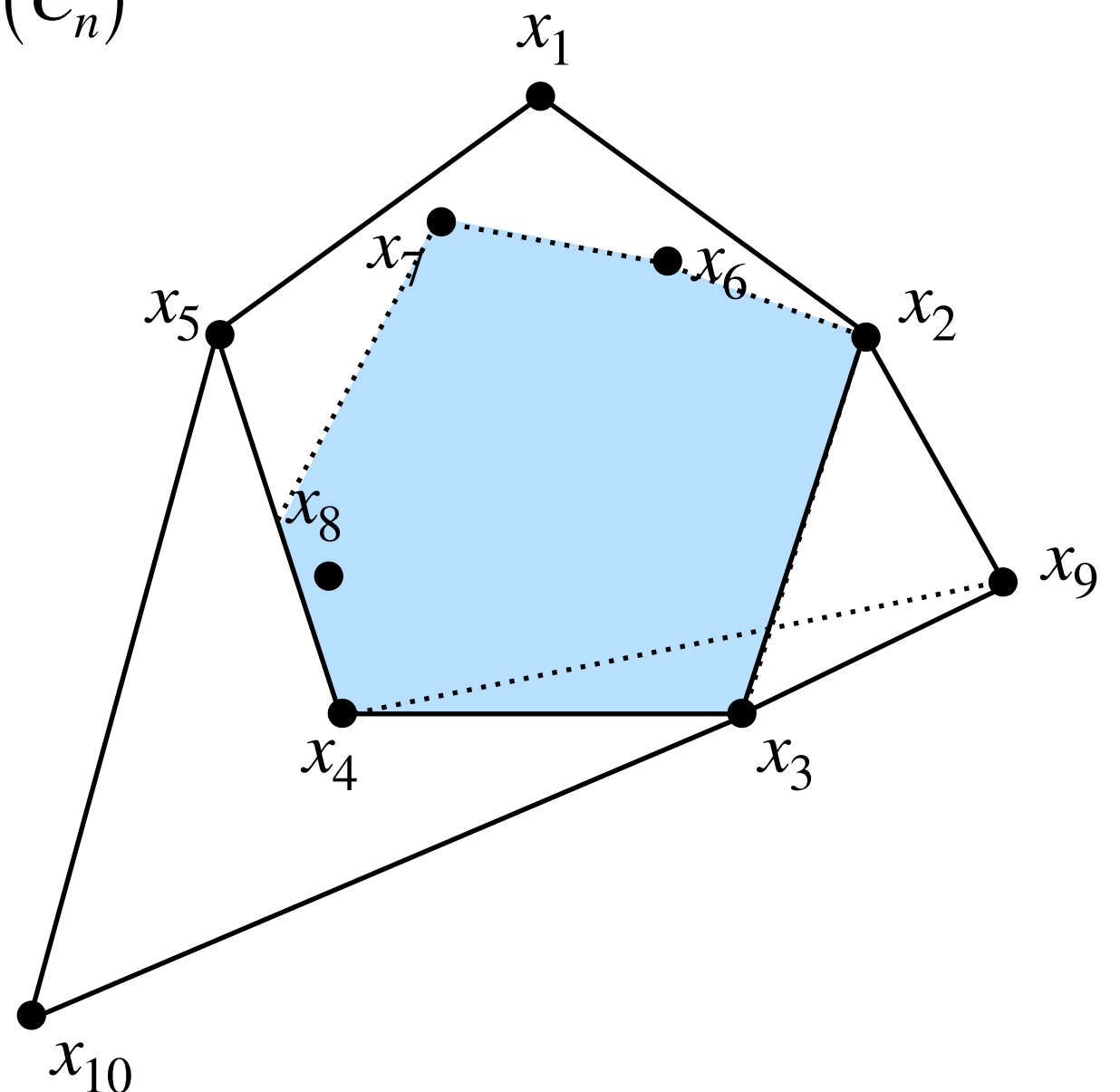
The Creative Process

- Let X_i be an i.i.d. random variable on $\mathbb{R}^d$
- For each $n \in \mathbb{N}$, define the random corpus $C_n = \{X_1, \dots, X_n\}$
- We want to understand $\lim_{n \rightarrow \infty} r_g(C_n)$



The Creative Process

- Let X_i be an i.i.d. random variable on $\mathbb{R}^d$
- For each $n \in \mathbb{N}$, define the random corpus $C_n = \{X_1, \dots, X_n\}$
- We want to understand $\lim_{n \rightarrow \infty} r_g(C_n)$



The Creative Process

- Let X_i be an i.i.d. random variable on $\mathbb{R}^d$
- For each $n \in \mathbb{N}$, define the random corpus $C_n = \{X_1, \dots, X_n\}$
- We want to understand $\lim_{n \rightarrow \infty} r_g(C_n)$
- It will be useful to write $X \in \mathbb{R}^d$ in terms of its polar coordinates:
 - Radial coordinate: $r = \|X\|$
 - Angular coordinate: $\theta = \frac{X}{\|X\|}$

Light-Tailed Creative Processes

Definition: The random variable $X \in \mathbb{R}$ with CDF F and density f has **light tails** if its hazard rate $h(t) = f(t)/(1 - F(t))$ is nondecreasing.

e.g., Gaussian distribution

Definition: The random vector $X \in \mathbb{R}^d$ has **uniform light tails** if:

1. $\|X\|$ has light tails
2. innovation happens in all directions: $\exists c > 0$ such that for all $r \geq 0$, the conditional density satisfies $f(\theta : \|X\| \geq r) \geq c$

e.g., standard multivariate Gaussian

Continuous Generation

Definition: The generator g is **uniform lower semicontinuous (ULS)** if for every $\varepsilon > 0$ there is a $\delta > 0$ such that

$$(1 - \varepsilon)g(C) \subseteq g((1 - \delta)C)$$

for all corpora $C \in \mathcal{C}$ where $0 \in C$

i.e., contracting the corpus slightly causes only a slight contraction in the generable set

Light-Tailed Creative Processes

Proposition: Fix a convex-valued and ULS generator g . If X_i has uniform light-tails, then

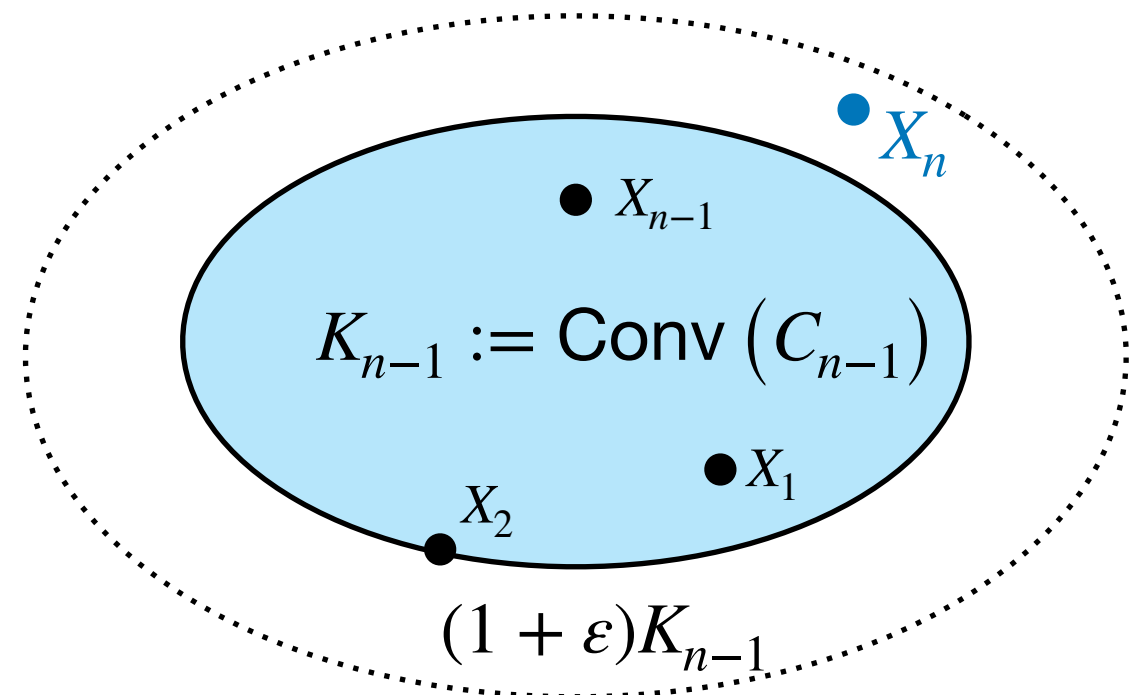
$$\lim_{n \rightarrow \infty} r_g(C_n) = 1 \quad \text{a.s.}$$

- In creative domains with gradual innovation, copyright infringement is a vanishing concern for large corpora.
- The violation set shrinks to measure zero.

Proof Idea

Part A: Show the result for the convex hull generator

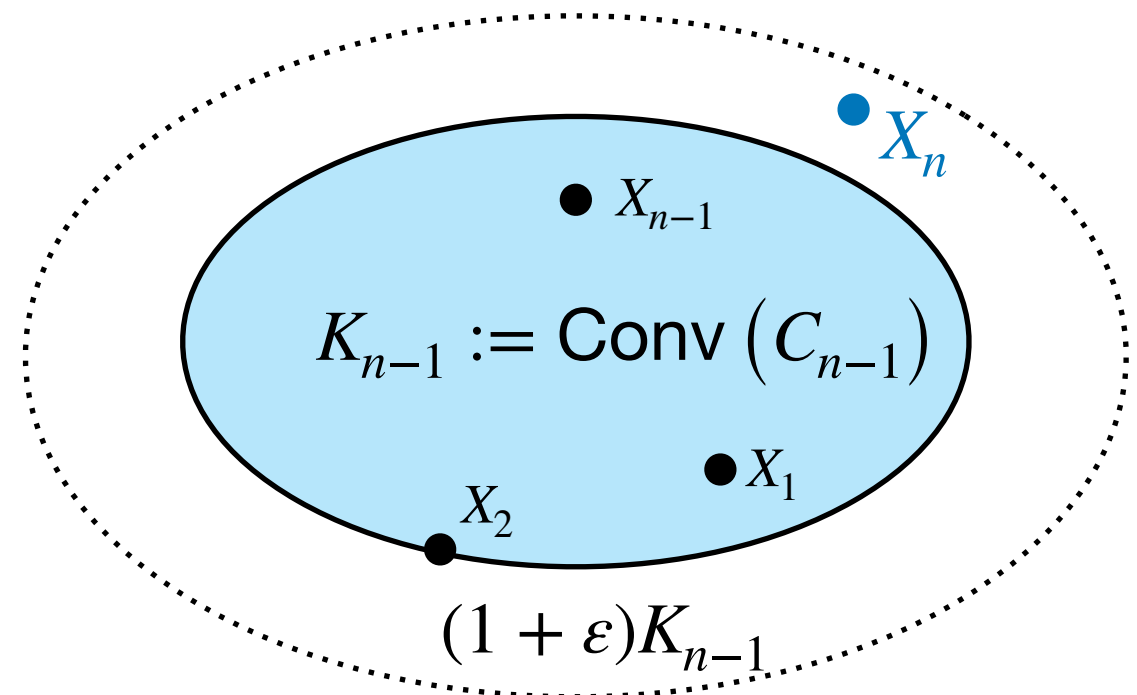
- Recall that an output is permissible only if it remains generable after removing any one work from the corpus.
- **Lemma:** for every $\varepsilon > 0$, a.s. there exists $N < \infty$ such that eventually every new work lies within a $(1 + \varepsilon)$ -expansion of the convex hull of the previous works



Proof Idea

Part A: Show the result for the convex hull generator

- Recall that an output is permissible only if it remains generable after removing any one work from the corpus.
- **Lemma:** for every $\varepsilon > 0$, a.s. there exists $N < \infty$ such that eventually every new work lies within a $(1 + \varepsilon)$ -expansion of the convex hull of the previous works
- Thus deleting X_n eventually has a small impact on the convex hull. We further argue that the effect of deleting any $X_1, \dots, X_{n-1}$ is also small.



Proof Idea

Part A: Show the result for the convex hull generator

- Recall that an output is permissible only if it remains generable after removing any one work from the corpus.
- **Lemma:** for every $\varepsilon > 0$, a.s. there exists $N < \infty$ such that eventually every new work lies within a $(1 + \varepsilon)$ -expansion of the convex hull of the previous works
- Thus deleting X_n eventually has a small impact on the convex hull. We further argue that the effect of deleting any $X_1, \dots, X_{n-1}$ is also small.

Part B: Extend the result for all convex-valued and ULS generators g

- Intuition: Once one-point deletion changes $\text{Conv}(C_n)$ only slightly, the same is true for $g(C_n)$

Heavy-Tailed Creative Processes

In contrast...

- Suppose X_i are Pareto distributed on $\mathbb{R}_+$ and let $g = g_{\text{conv}}$ be the convex hull generator.
- **Result:** The limiting permissible ratio $\lim_{n \rightarrow \infty} r_g(C_n)$ is bounded below 1

Takeaways

- In creative fields characterized by sufficiently frequent breakthroughs (such as avant-garde art and literary fiction)
 - protection for individual works may retain its importance even as corpora grow large
 - creators of paradigm-shifting works can continue to have legitimate infringement claims
- By contrast, in creative markets where innovation is largely incremental (possibly including genre fiction and commercial movies),
 - the marginal contribution of any single work diminishes rapidly as the corpus expands
 - generation is eventually unrestricted under our criterion

Extension: Protected Groups of Works

Generalization to Protected Groups

- In practice, copyright disputes often arise at the level of collections rather than single works



Generalization to Protected Groups

- In practice, copyright disputes often arise at the level of collections rather than single works
- Can generalize our criterion to handle this
- A *collection* of protected works is a family of subsets $\mathcal{A} = \{A_1, \dots, A_n\}$ where each $A_i \subseteq C$ is a **protected group**
 - Our main case: $\mathcal{A}$ includes all singleton sets from the corpus C

Generalization to Protected Groups

Definition:

The **permissible set** is

$$p_g^{\mathcal{A}}(C) := \bigcap_{A \in \mathcal{A}} g(C \setminus A)$$

The **violation set** is

$$v_g^{\mathcal{A}}(C) := g(C) \setminus p_g^{\mathcal{A}}(C).$$

all works that can be generated without any inputs from A



Generalization to Protected Groups

Definition:

The **permissible set** is

$$p_g^{\mathcal{A}}(C) := \bigcap_{A \in \mathcal{A}} g(C \setminus A)$$

The **violation set** is

$$v_g^{\mathcal{A}}(C) := g(C) \setminus p_g^{\mathcal{A}}(C).$$

all works that can be generated without any inputs from A



Generalization to Protected Groups

Definition:

The **permissible set** is

$$p_g^{\mathcal{A}}(C) := \bigcap_{A \in \mathcal{A}} g(C \setminus A)$$

all works that do not critically
rely on inputs from some $A \in \mathcal{A}$

The **violation set** is

$$v_g^{\mathcal{A}}(C) := g(C) \setminus p_g^{\mathcal{A}}(C).$$

Generalization to Protected Groups

Definition: A collection $\mathcal{A}$ is **stronger** than another collection $\mathcal{B}$ if for every $B \in \mathcal{B}$ there is an $A \in \mathcal{A}$ such that $B \subseteq A$

Result: Fix a generator g and corpus C . If a collection $\mathcal{A}$ is stronger than a collection $\mathcal{B}$ then

$$p_g^{\mathcal{A}}(C) \subseteq p_g^{\mathcal{B}}(C).$$

Superadditivity

Since $\{A \cup B\}$ is stronger than $\{A, B\}$...

Corollary: Fix a generator g and corpus C . For any two protected sets $A, B \in \mathcal{C}$,

$$p_g^{\{A,B\}}(C) \supseteq p_g^{\{A \cup B\}}(C)$$

with strict inclusion possible.

i.e., violations are **superadditive**: the violation of a union can exceed the union of their individual violations

Implication: when multiple creators join a class action, their combined violation set can exceed the sum of their individual claims

Related Literature

- We are most related to a small but growing set of papers in economics on copyright and intellectual property in the context of GenAI
 - Lutes (2025), de Rassenfosse et al. (2024), Gans (2024), Yang and Zhang (2025)
- Our criterion is spiritually similar to “leave-one-out” policies in machine learning and the Shapley value in game theory
- Our main result contributes to broader literature on the nature of creative processes and innovation
 - Dividends to creative work follow heavy-tailed distributions (Rosen (1981); Chung and Cox (1994); de Vany and Walls (1999))
 - The process of innovation itself may be heavy-tailed (Drugov and Ryvkin (2020), Lutes (2025))

Conclusion and Open Questions

- Our paper proposes a new definition for when it is the case that a generated work is “derivative” of an original
- We characterize the structure of permissible generation but takes as given both the generator and the distribution of creative works.
- A natural extension is to endogenize these objects.
- If creators anticipate how generative systems will use existing works, they may adjust what they produce.
- Will also be interesting to consider the coalitions that are incentivized to form.

Thank You